



ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY
MIJAR, MOODBIDRI - 574225, KARNATAKA

PROJECT REPORT

on

Dynamic Forecast of Expected Time of Arrival (ETA) for Coaching Trains

Data-driven arrival-time prediction for Indian Railways

A report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

SUJAN K S | 4AL23CS164

RAKESH K | 4AL23CS117

TEAM RAKESH

Under the Supervision of

ANJIT RAJA R

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025 – 2026



ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY
MIJAR, MOOBBIDRI - 574225, KARNATAKA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “**Dynamic Forecast of Expected Time of Arrival (ETA) for Coaching Trains**” has been successfully completed by **SUJAN K S (4AL23CS164)** and **RAKESH K (4AL23CS117)**, and is the work done by them and submitted during the 2025-26 academic year, in partial fulfillment for the award of the Degree of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi, during the year 2025-2026. It is certified that all corrections/suggestions indicated have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the award of the Bachelor of Engineering degree.

ANJIT RAJA R

Guide

Dr. Manjunath Kotari

HOD, CSE

2025 – 2026



ALVA'S INSTITUTE OF ENGINEERING AND TECHNOLOGY
MIJAR, MOODBIDRI - 574225, KARNATAKA

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible; success is the epitome of hard work and perseverance, but steadfast of all is encouraging guidance.

So, with gratitude we acknowledge all those whose guidance and encouragement served as beacon of light and crowned the effort with success. We would like to thank our guide, **Anjit Raja R**, Department of Computer Science & Engineering, for constantly supporting us throughout the completion of this project.

We sincerely thank **Dr. Manjunath Kotari**, Professor and Head, Department of Computer Science & Engineering, who has been the constant driving force behind the completion of the project.

We thank our beloved Principal for his constant help and support throughout.

We are indebted to the **Management of Alva's Institute of Engineering and Technology**, Mijar, Moodbidri for providing an environment which helped us in completing this project.

SUJAN K S
4AL23CS164

RAKESH K
4AL23CS117

Dynamic ETA Forecasting Engine

Data-driven arrival-time prediction for Indian Railways coaching trains

Project Documentation

Architecture | Methodology | Evaluation | API | User Interface | Operations Guide

Item	Detail
Organisation	Ministry of Railways (problem owner)
Category / Theme	Software / Smart Automation
Build window	3-day hackathon MVP
Core stack	Python, FastAPI, SQLAlchemy, XGBoost, Next.js, React, TypeScript
Report type	Extended project report with introduction and user-interface walkthrough
Document version	1.0 (September 2026)

Note: all evaluation figures in this document come from a synthetic train-running simulator. They demonstrate the method, not real Indian Railways operational performance.

Contents

1. Executive Summary	4
2. Introduction	5
3. Problem Statement and Objectives	8
4. Scope	9
5. System Architecture	11
6. Technology Stack	13
7. Repository Structure	15
8. Data Model and Simulator	16
9. Feature Engineering	18
10. Forecasting Methodology	21
11. Model Evaluation	25
12. API Reference	28
13. Live Data Integration	30
14. Frontend	31
15. User Interface Walkthrough	33
16. Demonstration Scenario	40
17. Testing and Quality Assurance	41
18. Installation and Operation	42
19. Limitations and Risks	44
20. Scalability and Extensibility	45
21. Innovation Highlights	46
22. Glossary	47
23. Source Documents	48
24. Conclusion	48
Appendix A. Worked Calculations	49
Appendix B. Troubleshooting Guide	50
Appendix C. Frequently Asked Questions	50
References	51

List of Figures

Figure 1. End-to-end architecture from data sources to consumers.	11
Figure 2. Simple flow: from raw train state to an ETA with a confidence band.	11
Figure 3. Core entities and how they relate (conceptual view).	16
Figure 4. The three seeded corridors (schematic; stations are evenly drawn and not to scale).	17
Figure 5. Feature importance by gain, on a logarithmic axis (initial single-segment model).	20
Figure 6. The residual target occupies a small, horizon-independent range, unlike the absolute target.	21
Figure 7. Segment chaining: predictions are summed and the state is propagated forward.	22
Figure 8. How the heuristic margin widens with the number of segments ahead.	23
Figure 9. Journey-level split used for training and evaluation.	23
Figure 10. Baseline versus ML: MAE on 40 held-out journeys (2,430 samples).	25
Figure 11. MAE by horizon and operating regime.	26
Figure 12. Baseline, first ML model and final ML model compared.	27
Figure 13. Live-data request with cache and simulator fallback.	30
Figure 14. Control Room: fleet table with baseline and ML ETAs for eight active simulated trains.	33
Figure 15. Control Room, left half: train, source, current station, speed and delay columns.	34
Figure 16. Control Room, right half: trend, next station, baseline ETA, ML ETA and action columns.	34
Figure 17. Passenger Lookup, steps 1 and 2: choosing a train and a destination station.	35
Figure 18. Zoom: current location, stops ahead and the destination selector.	35
Figure 19. Passenger Lookup, step 3: dynamic ETA forecast with the expected arrival window.	36
Figure 20. Zoom: estimated arrival 08:54, about 7 minutes of transit remaining and a range of 08:49 to 08:59.	36
Figure 21. Station Board for CNB (Kanpur Central): two inbound trains with ML ETAs and uncertainty ranges.	37
Figure 22. Zoom: expected arrival and uncertainty range, minutes remaining, status and Details action.	37
Figure 23. Model Evaluation: overall MAE and RMSE, followed by the prediction horizon breakdown.	38
Figure 24. Zoom: Mean Absolute Error card with baseline 74.11 min against XGBoost 28.27 min.	38
Figure 25. Zoom: Root Mean Squared Error card with baseline 102.01 min against XGBoost 48.09 min.	39
Figure 26. Zoom: horizon breakdown cards for one and three stations ahead.	39
Figure 27. Demo event loop: one injection updates every screen.	40

1. Executive Summary

Indian Railways publishes arrival estimates using the static timetable, the train's current delay, and a fixed recovery-time buffer. That approach ignores what is actually happening on the track ahead: congestion, unscheduled halts, temporary speed restrictions, weather, and knock-on effects from preceding trains. The result is an ETA that drifts from reality, which affects passengers, station operations, crew and platform planning, and feeder logistics.

The **Dynamic ETA Forecasting Engine** is a working end-to-end system that ingests a train's running state and continuously re-predicts its arrival time at every upcoming station. It always computes the traditional baseline alongside a machine-learning forecast, so the improvement is visible and measurable at all times.

1.1 Headline results

Table 1. Headline results on 40 held-out simulated journeys (MAE, minutes).

Horizon	Baseline MAE	ML MAE	Improvement
1 station ahead	66.47 min	5.17 min	+92.2%
3 stations ahead	78.83 min	43.53 min	+44.8%
5 stations ahead	87.32 min	66.34 min	+24.0%
Overall (2,430 samples)	74.11 min	28.27 min	+61.9%

1.2 What has been built

- A train-running simulator with kinematics, station dwell, and five injectable disruption types.
- A feature pipeline producing 11 model-ready features per train checkpoint.
- Two prediction paths: a deterministic baseline and an XGBoost residual model, chained segment by segment for multi-station ETAs with confidence bands.
- A FastAPI backend exposing train, ETA, station-arrival, simulation-control, demo and model-metrics endpoints.
- A Next.js frontend with a control-room dashboard, train detail view, passenger lookup, station arrivals board and a model-evaluation view.
- An optional live-data plug-in (RailRadar) with caching, rate-limit handling and automatic fallback to the simulator.
- An automated test suite of over 150 test functions across 16 modules.
- Innovations such as the schedule-residual target, always-on baseline comparison and safe live-data fallback (Section 21).

1.3 Honest framing

The training and evaluation data are synthetic. The value demonstrated here is the architecture, the modelling formulation, and the measurable gain over the baseline under controlled conditions. The system is designed so that a real data feed can replace the simulator without changing the model, API, or frontend contracts.

Reading the numbers responsibly

Every accuracy figure in this report is relative to the synthetic simulator. The baseline error of 66 to 87 minutes reflects the evaluation design and the simulated data. It should be read as a comparison between two methods under identical conditions, never as a statement about how accurate Indian Railways' published ETAs are in practice.

1.4 What is new in this edition

This edition extends the original project documentation with a full **Introduction** (Section 2), a set of worked calculation examples, additional charts, a detailed **user-interface walkthrough** built from live screenshots of the running application (Section 15), an operations troubleshooting guide, and appendices. All technical claims, metrics and design decisions remain those of the original document.

2. Introduction

2.1 Background

Railways are timetable-driven systems. Every passenger service is planned as a sequence of scheduled arrivals and departures at stations, and the rest of the operation, from platform allocation to crew duty rosters to the onward journeys of connecting passengers, is arranged around those planned times. When a train runs exactly to timetable, the schedule is also the best possible estimate of when it will arrive. The difficulty begins the moment a train deviates from the plan.

Long-distance coaching trains on busy trunk routes share track with other passenger and freight services. A train can lose time behind a slower service, be held at a signal, slow down for a temporary speed restriction, or be affected by fog or heavy rain. It can also recover time on clear sections, because timetables include margins that allow a late train to catch up. The true arrival time is therefore a function of what happens between the train's current position and its destination, and not only of the plan.

2.2 Why arrival-time prediction matters

An estimated time of arrival is not just information for a passenger waiting on a platform. It is an input to many decisions taken by different people at the same time.

Table 2. Consumers of arrival-time estimates and the decisions they take.

Who	Decision that depends on the ETA
Passengers	When to leave home, when to reach the station, whether a connection is still feasible, and whom to inform about a late arrival.
Station operations	Which platform to plan for, when to position staff, and how to sequence the arrivals board.
Control room	Where delays are building up, and how a disruption on one section will ripple into the stations downstream.
Crew and platform planners	Relieving crews, cleaning and turnaround slots, and platform occupancy.
Feeder logistics	Pickup and drop-off timing for taxis, buses and other onward transport.

A consistently inaccurate ETA erodes trust. Passengers learn to ignore it, and operations teams learn to correct it by experience. A forecast that is *measurably* better than the default, and that honestly expresses how uncertain it is, is valuable to all of these groups.

2.3 How ETAs are estimated today

The current practice can be summarised in one line: **scheduled arrival plus the current delay, minus a recovery buffer built into the timetable**. The rule is simple, transparent and always available. Its weakness is that it treats the future as a straight-line extension of the present. It assumes today's delay simply carries forward and that recovery follows a fixed schedule margin, regardless of what is happening ahead.

In practice, trains recover time on clear sections, lose time behind slower traffic, and are throttled by speed restrictions and weather. A single static rule cannot represent all of that. This gap between the static rule and operational reality is the opportunity that this project addresses.

2.4 The idea behind this project

The project starts from a modest observation. Most of a journey's remaining time is already captured by the timetable. What the timetable cannot predict is the *operational deviation*: how much faster or slower than plan the train will be over the remaining distance. If a model is asked to predict only that deviation, the problem becomes far more tractable than predicting raw minutes from scratch.

The engine therefore anchors its forecast on the timetable and learns only the residual. It then chains predictions segment by segment so that a single small model can forecast every downstream station, and it attaches a confidence band that widens with the forecast horizon. Everything is surfaced through one API, so that a dashboard, a passenger screen and a station board show exactly the same numbers.

2.5 Hackathon context

Table 3. Hackathon context.

Attribute	Detail
Event	Smart India Hackathon
Problem statement	26028: Dynamic ETA Forecasting
Problem owner	Ministry of Railways
Category / theme	Software / Smart Automation
Build window	3-day hackathon MVP
Deliverable	A working end-to-end prototype, an evaluation against the current method, and a scalability argument

Because the build window was three days, the design deliberately favours a small number of well-understood components. SQLite and in-process caching replace heavier infrastructure, a synthetic simulator replaces real telemetry, and a single gradient-boosted model replaces a more elaborate architecture. These choices keep the prototype reproducible and easy to demonstrate, while the interfaces are drawn so that real components can be substituted later.

2.6 Contributions of this work

- 1 **A schedule-residual formulation** that keeps the regression target within a stable range (about -90 to +60 minutes) irrespective of how far ahead the forecast looks.

- 2 **A segment-chaining predictor** that produces an ETA and a confidence band for every downstream station from one small model.
- 3 **A source-agnostic architecture** in which the simulator and a live API both emit the same normalised state, so that no downstream component knows which one is in use.
- 4 **An always-on baseline comparison** that makes the improvement visible for every station and guarantees an answer even without the model.
- 5 **An honest, leakage-safe evaluation** reported by horizon and by disruption regime, including the early failures and how they were fixed.
- 6 **A complete demonstrable product:** API, four frontend views, disruption injection and a deterministic demo mode.

2.7 Organisation of this report

Table 4. How this report is organised.

Sections	Content
1 to 4	Executive summary, introduction, problem statement and objectives, and scope.
5 to 8	System architecture, technology stack, repository structure, data model and simulator.
9 to 11	Feature engineering, forecasting methodology and model evaluation.
12 to 14	API reference, live data integration and frontend design.
15 to 16	User-interface walkthrough with screenshots, and the demonstration scenario.
17 to 19	Testing, installation and operation, limitations and risks.
20 to 24	Scalability, innovation highlights, glossary, sources and conclusion.
Appendices	Worked examples, quick-reference tables, troubleshooting, FAQ.

2.8 Conventions used in this report

- All error values are in **minutes**. MAE is mean absolute error and RMSE is root mean squared error.
- Station codes (for example **NDLS**, **CNB**, **HWH**) follow the codes used in the application.
- Code, file names and API endpoints are shown in a monospaced font, for example `backend/main.py`.
- Statements about delivered functionality are separated from proposals; proposals are always labelled as not built.
- Figures marked schematic are illustrations of structure and are not drawn to scale.

3. Problem Statement and Objectives

3.1 The problem

The current ETA method can be summarised as scheduled arrival plus current delay minus a recovery buffer built into the timetable. It assumes that today's delay simply carries forward and that recovery follows a fixed schedule margin. In practice, trains recover time on clear sections, lose time behind slower traffic, and are throttled by speed restrictions and weather. A single static rule cannot represent that.

3.2 Where a static rule breaks down

Table 5. Situations where a static schedule-plus-delay rule diverges from reality.

Situation on the track	What the static rule assumes	What may actually happen
Clear section ahead	Recovery follows the fixed timetable margin.	The train may recover more than the buffer allows.
Slower traffic ahead	Current delay simply carries forward.	The delay may grow as the train is held behind traffic.
Temporary speed restriction	No effect on the remaining time.	Additional minutes are lost on the affected segment.
Fog or heavy rain	No effect on the remaining time.	Speed is dampened over the section.
Unscheduled halt	Not represented.	Delay accumulates while the train is stationary.

3.3 Objectives

- 1 Build a model that **measurably beats** the naive baseline on held-out journeys, reported as MAE and RMSE in minutes.
- 2 **Update ETAs dynamically** as a train's position and events change, without restarting anything.
- 3 Expose a **REST API** that a dashboard, a passenger-style screen and a station-board view all consume live.
- 4 Present a **clear story**: problem, data, model, live demo, and scalability argument.

3.4 Stakeholders and consumers

Table 6. Stakeholders and consumers.

Consumer	What they need from the ETA
Passengers	A reliable arrival time and an honest confidence range for a chosen train and station.
Station operations	An arrivals board sorted by predicted, not scheduled, arrival.
Control room	Fleet-wide view of delays, plus the ability to see how disruptions ripple into downstream ETAs.
Downstream planners	Crew, platform and feeder-logistics systems that consume the same API (described, not built).

3.5 Success criteria

The objectives translate into criteria that can be checked directly against the running system and the evaluation report.

Table 7. Success criteria and evidence.

Objective	How success is judged	Where evidenced
Beat the baseline	Lower MAE and RMSE than the baseline on the same held-out journeys, at every horizon.	Section 11
Dynamic updates	ETAs change on every screen after an event is injected, without a restart.	Sections 15, 16
Live API for all views	Dashboard, passenger and station views read the same endpoints.	Sections 12, 14
Clear story	Five-minute demonstration with reproducible numbers.	Section 16

3.6 Working assumptions

- A train's running state (position, speed, delay, events) is available at regular intervals, either from the simulator or from a live feed.
- The timetable for each route, including cumulative distance and scheduled arrival and departure at each station, is available.
- Disruptions can be represented by a small set of event types with a bounded effect on speed and delay.
- Because no real running logs were available for training, a realistic synthetic simulator is an acceptable stand-in for demonstrating the method.

4. Scope

4.1 In scope (delivered)

- Synthetic but realistic train-running simulator with multi-station routes and injected disruptions.
- Feature-engineering pipeline from raw running state to model-ready features.
- Baseline heuristic and gradient-boosted ML predictor, with multi-station chaining and confidence intervals.
- FastAPI backend with prediction and simulation-control endpoints.
- Dashboard, train detail, passenger lookup and station arrivals board in the frontend.
- Evaluation report: MAE and RMSE versus baseline, by horizon and by disruption regime.
- Optional live-data adapter (RailRadar), treated as a plug-in and never a dependency.

4.2 Explicitly out of scope

- Integration with real NTES or Control Office Application systems. A marked plug-in point exists instead.
- GPS or IoT hardware ingestion. A simulated position feed is used.
- Crew scheduling, platform allocation and cleaning-operations optimisation.
- Authentication, multi-tenant zone management, horizontal scaling and formal load testing.
- Deep-learning sequence models (LSTM or Transformer), deferred as a stretch goal.

4.3 Scope boundary in one view

Table 8. Delivered versus future work.

Area	Delivered in this build	Left as future work
Data	Synthetic simulator, optional RailRadar on-demand lookup	Real running logs, NTES / COA feeds, GPS or IoT ingestion
Model	XGBoost residual model with segment chaining	Sequence models, cross-train effects, per-zone models
Uncertainty	Heuristic band, $5.0 \times \sqrt{\text{segments ahead}}$	Calibrated quantile or conformal intervals
Serving	FastAPI, SQLite, in-process cache	Authentication, observability, horizontal scaling, load testing
Downstream	Same API available to consumers	Crew, platform and connection-risk integrations

Downstream planner integrations are described in the product requirements but are not built. They are listed here so that the boundary is unambiguous.

5. System Architecture

The system is a linear pipeline. Data sources produce a normalised train state; the feature builder converts that state into model inputs; two predictors sit behind a single API; and the front-end views read from that API. There is exactly one source of truth for every consumer.

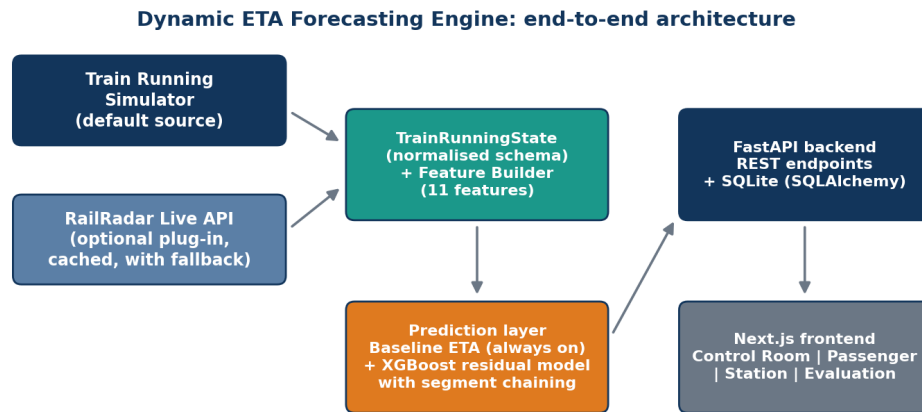


Figure 1. End-to-end architecture from data sources to consumers.

5.1 Simple flow of one prediction

Every ETA the system shows follows the same six steps, whether the train state came from the simulator or from the live API.

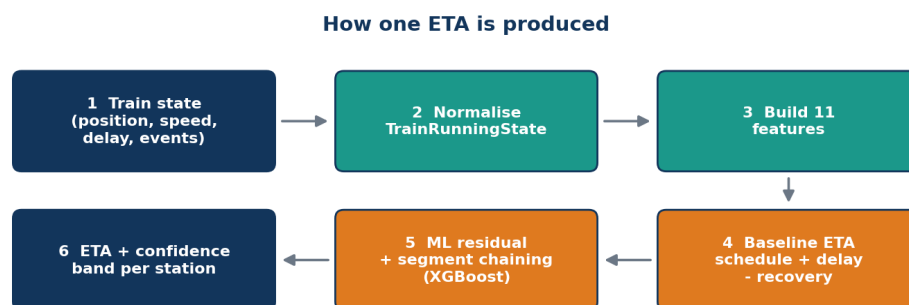


Figure 2. Simple flow: from raw train state to an ETA with a confidence band.

Table 9. The six steps of a single prediction.

Step	What happens	Produced by
1. Train state	Position, speed, delay and active events are read for the train.	Simulator or RailRadar adapter
2. Normalise	The state is expressed as a TrainRunningState object.	Data-source layer
3. Build features	Eleven model-ready features are computed from the state and timetable context.	backend/features/
4. Baseline ETA	Scheduled arrival plus delay minus recovery buffer.	backend/services/
5. ML residual and chaining	XGBoost predicts the residual per segment; segments are chained forward.	backend/ml/ and services

Step	What happens	Produced by
6. ETA and band	A predicted ETA, baseline ETA and confidence band per station is returned.	FastAPI endpoints

5.2 Design principles

- **Source-agnostic core.** Simulator and live API both emit the same `TrainRunningState` object, so nothing downstream knows which one is in use.
- **Baseline is always available.** It is computed even if the ML model is missing, so the API never returns nothing.
- **No hidden infrastructure.** SQLite and in-process caching replace Kafka, Redis and managed databases.
- **Fail soft.** Live-API errors (timeout, 401, 404, 429, 503, malformed payload) fall back to the simulator with clear provenance flags.

Why each principle matters

Source-agnostic core. The most valuable property of the design is that the model, the feature code, the API and the user interface never need to know where a train's state came from. Replacing the simulator with a real feed is therefore an adapter task, not a rewrite. It also means that the live-data path is tested by the same code that serves the simulator.

Baseline always available. A forecasting service that returns nothing when a component fails is worse than one that returns a less sophisticated answer. Because the baseline depends only on the timetable, the current delay and the recovery buffer, the API can always answer, and the interface can always show the comparison between the current method and the model.

No hidden infrastructure. A prototype that needs a message broker, a cache server and a managed database is hard to run on a laptop and hard to demonstrate. Using SQLite through SQLAlchemy and a thread-safe in-process cache keeps the system to a handful of processes, while the repository layer leaves room to swap the storage later.

Fail soft. Third-party services time out, rate-limit and change. Each failure class is mapped to a typed error and a controlled fallback, and every response carries flags saying where the data came from, so that a fallback is visible rather than silent.

5.3 Backend module responsibilities

Table 10. Backend modules and their responsibilities.

Module	Responsibility
<code>backend/main.py</code>	FastAPI entry point: CORS, table initialisation, router registration, simulator lifecycle, all endpoints.
<code>backend/database/</code>	SQLAlchemy models (stations, routes, route_stations, trains, journeys, events), connection handling, repository helpers, corridor seed data.
<code>backend/simulator/</code>	Discrete-event engine, per-train journey kinematics, and the five disruption event types.
<code>backend/features/</code>	FeatureBuilder producing the 11 PRD features, cumulative schedule computation and delay-trend calculation.
<code>backend/ml/</code>	Dataset generator, XGBoost training, evaluation module, predictor and model metadata.

Module	Responsibility
<code>backend/services/</code>	Baseline ETA, multi-station ETA chaining, data-source abstraction, RailRadar client, cache, live-train lookup, demo scenario, shared schemas.

5.4 Runtime behaviour

At start-up the API initialises the database tables, registers its routers and starts the simulator lifecycle, after which the simulator advances every train on a fixed tick. On each tick a normalised `TrainRunningState` is produced for each running train. When a client asks for an ETA, the service builds features from the latest state, computes the baseline, applies the ML predictor segment by segment, and returns the result together with confidence bounds.

Nothing in this loop needs a restart when conditions change. An injected disruption alters the simulator's state for the affected train; the very next request builds features from the new state and returns an updated ETA. This is what allows all views to refresh after a single event.

5.5 Failure modes and responses

Table 11. Failure modes and controlled responses.

Condition	Response of the system	What the user sees
ML model file missing	Baseline ETA is still computed and returned.	A baseline-only answer; the API never returns nothing.
No RailRadar API key	Effective data-source mode remains SIMULATOR.	Simulator telemetry, as in the default mode.
Live API timeout, 401, 404, 503, malformed payload	Typed error and fallback to simulator state.	ETA tagged with provenance flags.
Live API HTTP 429	Cooldown; stale cached response may be served.	Response flags <code>is_cached</code> and <code>is_stale</code> .
Fresh cache hit	Cached state is used, upstream call avoided.	Response flags <code>cached_at</code> and <code>age_seconds</code> .

6. Technology Stack

Table 12. Technology stack.

Layer	Technology	Why
Language	Python 3; TypeScript	Team comfort zone; strong data and ML ecosystem; typed frontend.
API	FastAPI, Uvicorn, Pydantic	Typed request and response contracts, automatic OpenAPI docs, fast to build.
Data handling	pandas, numpy	DataFrame pipelines, vectorised feature maths.
Persistence	SQLite via SQLAlchemy	Zero-setup store for a hackathon MVP.
ML	XGBoost	Strong on tabular data, small and cheap to serve.
HTTP client	httpx	Live-API adapter with timeouts and error mapping.

Layer	Technology	Why
Configuration	python-dotenv	Environment-only handling of secrets such as the RailRadar key.
Testing	pytest	Unit and integration coverage across modules.
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4	Modern component model, responsive layouts, typed API client.

6.1 Backend technologies

FastAPI with Uvicorn and Pydantic provides typed request and response contracts and generates interactive OpenAPI documentation at `/docs` without additional work. Because the same Pydantic models are mirrored as TypeScript interfaces in the frontend, a change to a response shape is caught at compile time on the client.

pandas and numpy carry the data pipelines: the dataset generator, the feature builder and the evaluation module all work on DataFrames with vectorised arithmetic. **SQLAlchemy over SQLite** gives a zero-setup persistence layer that is nevertheless expressed in standard ORM models, so a move to a server database would not change the model definitions.

httpx is used for the live-API adapter because it makes timeouts explicit and lets HTTP failures be mapped to typed errors. **python-dotenv** keeps secrets, such as the RailRadar key, in the environment and out of version control.

6.2 Machine-learning technology

XGBoost is well suited to this problem for three reasons stated in the project: it performs strongly on tabular data, the trained model is small, and inference is cheap. A single model applied per segment, rather than a model per train or per station, keeps serving cost low as the number of trains grows.

6.3 Frontend technologies

The frontend is a **Next.js 16 App Router** application written in **TypeScript** with **React 19** and **Tailwind CSS 4**. It only presents data; all business and prediction logic stays in the backend. A shared typed client lives in `frontend/src/lib/api.ts`, and interface definitions in `frontend/src/types/api.ts` mirror the backend Pydantic models.

6.4 Testing technology

pytest drives unit and integration tests across sixteen modules covering the API contract, the baseline and multi-station ETA, the features and dataset, the model and evaluation, the simulator and database, the live data path and the demo mode (see Section 17).

7. Repository Structure

```
dynamic-eta-forecasting/
|-- backend/
|   |-- api/           route definitions and schema contracts
|   |-- database/      models, connection, repository, seed data
|   |-- simulator/     engine, journey kinematics, events
|   |-- features/      feature_builder.py
|   |-- ml/            dataset_generator, train_model, evaluate_model, predictor
|   |-- services/      baseline, multi-station ETA, live data, cache, demo
|   |-- main.py        FastAPI application
|-- frontend/          Next.js application (src/app, components, lib, types)
|-- data/
|   |-- raw/           raw external datasets and simulator logs
|   |-- processed/     feature datasets and metrics JSON
|   |-- models/        serialised XGBoost model and metadata
|-- tests/             16 pytest modules
|-- notebooks/         exploratory analysis
|-- docs/              PRD and phase reports
|-- .env.example
|-- requirements.txt
|-- README.md
```

7.1 Directory guide

Table 13. Repository directories.

Directory	Purpose	Notes
backend/	All server-side code	Prediction, simulation, storage and API live here.
frontend/	Next.js application	Presentation only; no business logic.
data/raw/	Raw external datasets and simulator logs	Tracked with placeholder files.
data/processed/	Feature datasets and metrics JSON	Regenerated by the modules in backend/ml/.
data/models/	Serialised XGBoost model and metadata	eta_xgboost_model.json and eta_xgboost_metadata.json.
tests/	16 pytest modules	Run with python -m pytest.
notebooks/	Exploratory analysis	Not needed to run the system.
docs/	PRD and phase reports	See Section 23.

7.2 Where to look for what

- To change how an ETA is computed, start in **backend/services/** (baseline and chaining) and **backend/ml/** (predictor).
- To change which inputs the model sees, start in **backend/features/feature_builder.py**.
- To change how trains move or how disruptions behave, start in **backend/simulator/**.
- To add or change an endpoint, start in **backend/main.py** and the schema contracts in **backend/api/**.
- To connect a new data source, add an adapter that emits **TrainRunningState** (see Section 13).

8. Data Model and Simulator

8.1 Core entities

Table 14. Core entities.

Entity	Meaning
Station	A named station with a code (for example NDLS, CNB, HWH).
Route	An ordered list of stations with scheduled distance and time.
RouteStation	A station's position within a route: sequence, cumulative distance, scheduled arrival and departure.
Train	A train number and name attached to a route.
Journey	One running instance of a train on a date.
Event	A timestamped disruption attached to a journey.

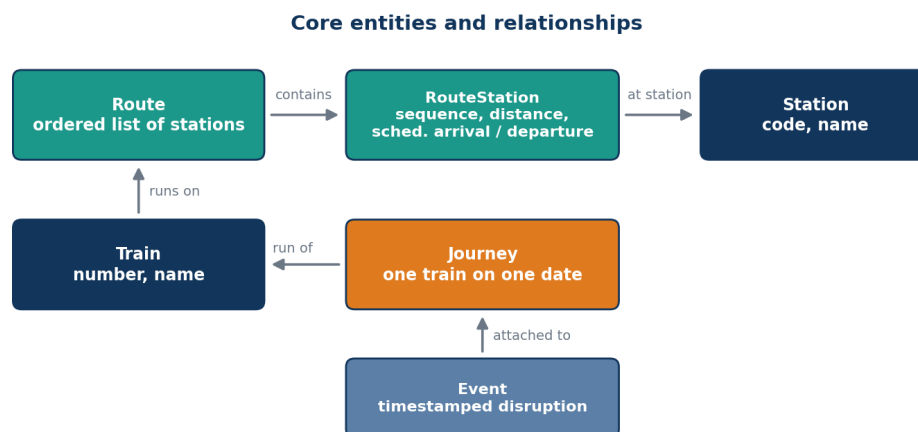


Figure 3. Core entities and how they relate (conceptual view).

8.2 Seeded corridors and trains

The synthetic dataset covers three trunk corridors and eight trains.

Table 15. Seeded corridors and trains.

Corridor	Stations (code)	Length	Trains
Eastern Trunk	NDLS, CNB, PRYJ, DDU, GAYA, DHN, ASN, HWH	1,447 km	12302 Howrah Rajdhani; 12260 Duronto; 12306 Kolkata Rajdhani (via Patna)
Western Trunk	NDLS, MTJ, KOTA, RTM, BRC, ST, MMCT	1,386 km	12952 Mumbai Rajdhani; 12926 Paschim Express; 12954 August Kranti Tejas Rajdhani
Southern Corridor	SBC, BNC, KJM, BWT, KPD, MAS	359 km	12028 Shatabdi; 12610 Chennai Express

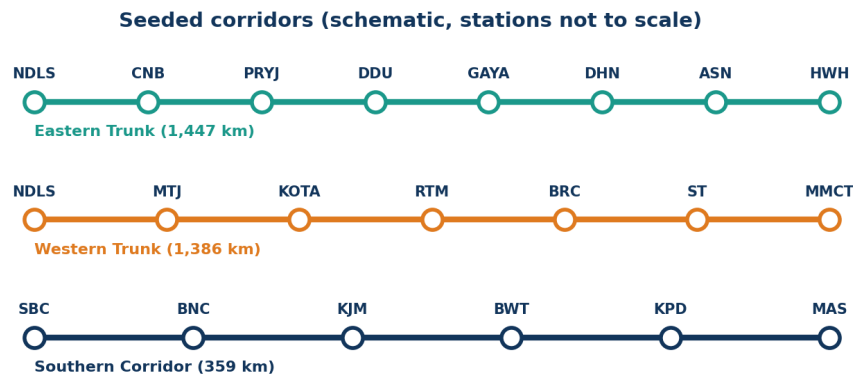


Figure 4. The three seeded corridors (schematic; stations are evenly drawn and not to scale).

8.3 The simulator

The simulator advances each train along its route on a tick. It applies acceleration, braking, station dwell times and a maximum speed of 110 km/h, and it emits a normalised `TrainRunningState` at each step. Simulation speed is configurable, from 1x real time up to 60x, or roughly one simulated minute per second.

Table 16. Simulator parameters.

Parameter	Value	Source
Maximum speed	110 km/h	Simulator kinematics
Tick interval (default)	2.0 real seconds	SIMULATION_TICK_SECONDS
Simulated time per tick (default)	2.0 minutes	SIMULATED_MINUTES_PER_TICK
Simulation speed range	1x to 60x	Configurable
Demo random seed	42	Deterministic demo mode

Disruption events

Table 17. The five disruption event types.

Event type	Effect
<code>SIGNAL_HALT</code>	Train stops; delay accumulates while halted.
<code>CONGESTION</code>	Reduced effective speed and higher downstream congestion score.
<code>SPEED_RESTRICTION</code>	Temporary speed restriction active on the upcoming segment.
<code>UNSCHEDULED_HALT</code>	Unplanned stop adding delay.
<code>WEATHER</code>	Fog or heavy rain flag with speed dampening.

These five types map directly onto features the model sees: a restriction sets `active_speed_restriction_flag`, congestion raises `congestion_score_downstream`, weather sets `weather_flag`, and halts show up through `current_delay_min` and `delay_trend_3pt`. That link is what allows the model to react when an event is injected.

8.4 The normalised running state

Both the simulator and the live API produce a **TrainRunningState**. This single schema is what lets the two data sources be interchangeable.

Table 18. Contents of the normalised running state.

Field group	Content
Identity	Train number and journey date.
Status	Running status (for example RUNNING or HALTED).
Location	Current station, next station, distance to the next station, and segment progress.
Motion	Speed and delay.
Provenance	Timestamp and a source tag of either simulator or external_api.

8.5 Training dataset

Table 19. Training dataset summary.

Metric	Value
Rows (observation checkpoints)	6,125
Journeys	200 (25 synthetic dates x 8 trains)
Columns	20 (9 identifier/metadata + 11 features)
Missing values	0 (100% complete)
Samples with a disruption	30.37%
Correlation of target with distance_to_go_km	r = 0.9837
Correlation of target with scheduled_time_to_go_min	r = 0.9633

The strong correlation of the (absolute) target with remaining distance and scheduled time is expected: most of a journey's remaining time is determined by how far the train still has to go. This is precisely the observation that motivates the schedule-residual formulation in Section 10.

9. Feature Engineering

The FeatureBuilder converts a running state and timetable context into eleven features per train per checkpoint.

Table 20. The eleven model features.

#	Feature	Group	Description
1	distance_to_go_km	Static	Remaining track distance to the target station.
2	scheduled_time_to_go_min	Static	Timetable-implied remaining minutes.
3	num_intermediate_halts	Static	Scheduled stops between the train and the target.
4	current_delay_min	Dynamic	Delay at the last reporting checkpoint.
5	delay_trend_3pt	Dynamic	Slope of delay across the last three checkpoints.

#	Feature	Group	Description
6	active_speed_restriction_flag	Dynamic	1 if a restriction is active on the upcoming section.
7	congestion_score_downstream	Dynamic	Traffic-density proxy for the section ahead, 0 to 1.
8	hist_avg_delay_this_section	Historical	Section mean historical delay (synthetic default).
9	hist_recovery_rate_section	Historical	Typical minutes recovered per 100 km (synthetic default).
10	weather_flag	Context	1 if fog or heavy precipitation caution is active.
11	day_type	Context	0 weekday, 1 weekend, 2 holiday.

9.1 Feature groups explained

Static features describe the journey ahead and do not change with events: how far the train has to go, how many minutes the timetable allots for that distance, and how many scheduled stops lie in between. They give the model the geometry of the problem.

Dynamic features capture what is happening now. The current delay says how far behind (or ahead) the train is; the three-point delay trend says whether that delay is growing or shrinking; the restriction flag and congestion score describe the section immediately ahead. These are the features that respond when a disruption is injected.

Historical features are intended to carry per-section, per-weekday behaviour: the typical delay on a section and the typical recovery rate per 100 km. In this build they use synthetic defaults, and are therefore not yet informative.

Context features capture conditions that modify speed without appearing in the train's own telemetry: whether fog or heavy precipitation caution is active, and whether the day is a weekday, weekend or holiday.

Data note

The two historical features currently use Day 1 synthetic defaults, because no real historical logs are available. With real running logs these would become genuine per-section, per-weekday statistics supplied through the SectionStatsProvider.

9.2 Feature importance (initial model)

Table 21. Feature importance from the first single-segment model.

Feature	Gain	Splits	Role
distance_to_go_km	62,477.77	682	Primary kinematic determinant of travel time
scheduled_time_to_go_min	34,957.09	563	Timetable foundation
congestion_score_downstream	654.94	215	Downstream traffic throttling
current_delay_min	564.94	386	Accumulated running delay
weather_flag	529.07	148	Rain and fog speed dampening
day_type	439.42	276	Weekday vs weekend vs holiday adjustment

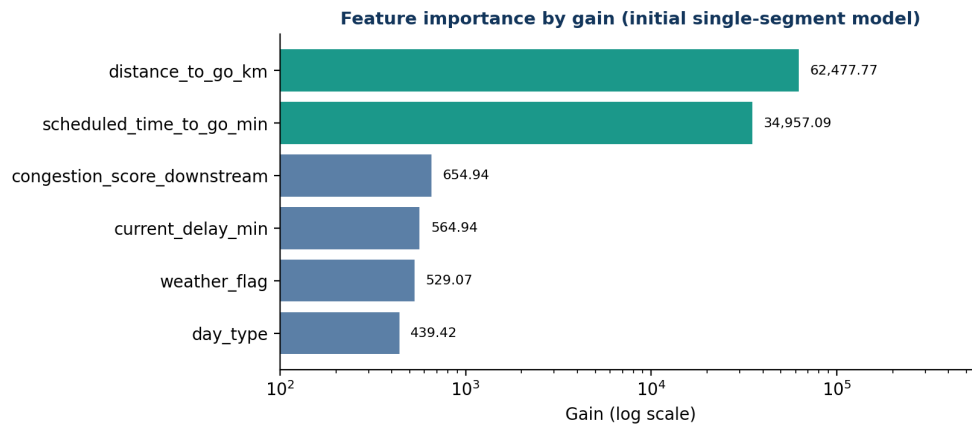


Figure 5. Feature importance by gain, on a logarithmic axis (initial single-segment model).

Importances come from the first single-segment model (absolute target). Distance and schedule dominate, as expected.

The importance table shows two orders of magnitude between the two kinematic features and the rest. That gap is the same fact seen through a different lens: predicting absolute remaining minutes is mostly a matter of knowing distance and schedule. Once the timetable is used to anchor the target (Section 10), the model is free to spend its capacity on the smaller operational signals: congestion, delay, weather and day type.

10. Forecasting Methodology

10.1 Baseline ETA

$\text{ETA} = \text{scheduled arrival} + \text{current delay} - \text{scheduled recovery buffer}$

This mirrors current practice. The recovery buffer defaults to 2.5 minutes per 100 km, a standard Indian Railways recovery margin, and predictions are guarded by a physical feasibility ceiling of 130 km/h. It is always computable, with no dependency on the ML model.

Worked example (illustrative numbers)

A train is scheduled to arrive at 10:00, is currently 14 minutes late, and is 195 km from the target. The recovery buffer is $2.5 \times 1.95 = 4.875$ minutes. Baseline ETA = 10:00 + 14 min - 4.9 min, that is about 10:09. The rule then says the delay will shrink by the fixed buffer, whatever is happening on the track ahead.

10.2 ML model: schedule-residual formulation

The first version predicted remaining minutes directly. It performed well on one station ahead but failed at longer horizons: tree models cannot extrapolate beyond the target range they saw in training. Single-segment targets spanned roughly 1 to 313 minutes, so predictions for stations 600 to 1,200 minutes away plateaued near 310 minutes.

The fix was to change the target so it stays in a stable range regardless of horizon:

```
target      = actual_remaining_minutes - scheduled_time_to_go_min
prediction   = scheduled_time_to_go_min + model_residual
```

The timetable anchors the large, linear part of the travel time, and XGBoost only has to predict the operational deviation, which sits in about -90 to +60 minutes across all horizons. Predictions are also clamped to be non-negative.

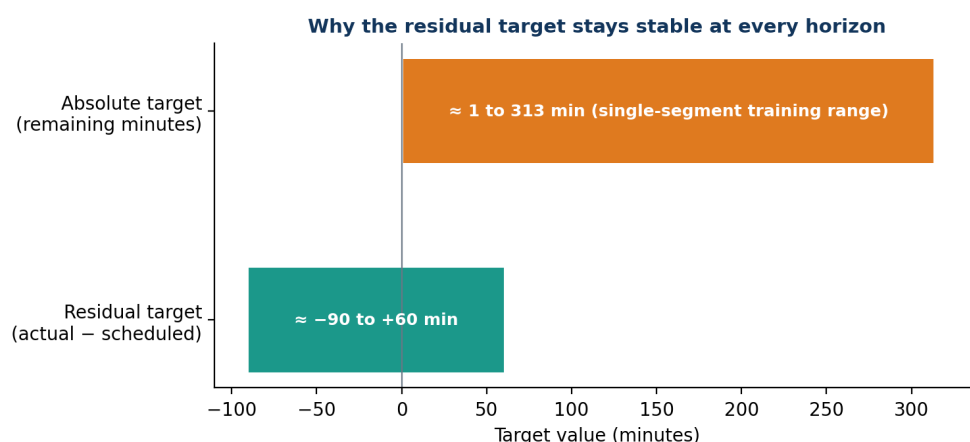


Figure 6. The residual target occupies a small, horizon-independent range, unlike the absolute target.

Hyperparameters

Table 22. XGBoost hyperparameters.

Parameter	Initial	Optimised (residual)
Objective	reg:squarederror	reg:squarederror
Boosting rounds	150	200
Learning rate	0.05	0.08
Max depth	5	6
Subsample / colsample by tree	0.85 / 0.85	Not restated in report
Random seed	42	42

10.3 Multi-station chaining

For every upcoming station on the route, the service predicts the transit time of the current segment, then forward-propagates the train state (arrival timestamp, accumulated delay, and scheduled dwell) into the next segment. Segment predictions are summed, so a single small model yields an ETA for every downstream station. Each station returns the predicted ETA, baseline ETA, confidence bounds and the segment-by-segment breakdown.

Segment chaining: one small model, every downstream station



state (arrival time, delay, dwell) is propagated forward into each next segment; predictions are summed

Figure 7. Segment chaining: predictions are summed and the state is propagated forward.

Worked example (illustrative numbers)

For a station three segments ahead, the service predicts segment 1 (say 42 min), advances the arrival time and delay, predicts segment 2 (say 55 min) from the updated state, then segment 3 (say 61 min). The ETA at the third station is the current time plus the three predictions plus any scheduled dwell, and the confidence margin is $5.0 \times \sqrt{3}$, about 8.7 minutes.

10.4 Confidence interval

The confidence band uses a transparent heuristic that widens with the forecast horizon:

margin = 5.0 x sqrt(segments_ahead) minutes

The 5-minute base comes from the single-segment test MAE (about 5.2 minutes).

Table 23. Confidence margin by number of segments ahead.

Segments ahead	1	2	3	4	5
Margin (min)	5.00	7.07	8.66	10.00	11.18
Band width (min)	10.0	14.1	17.3	20.0	22.4

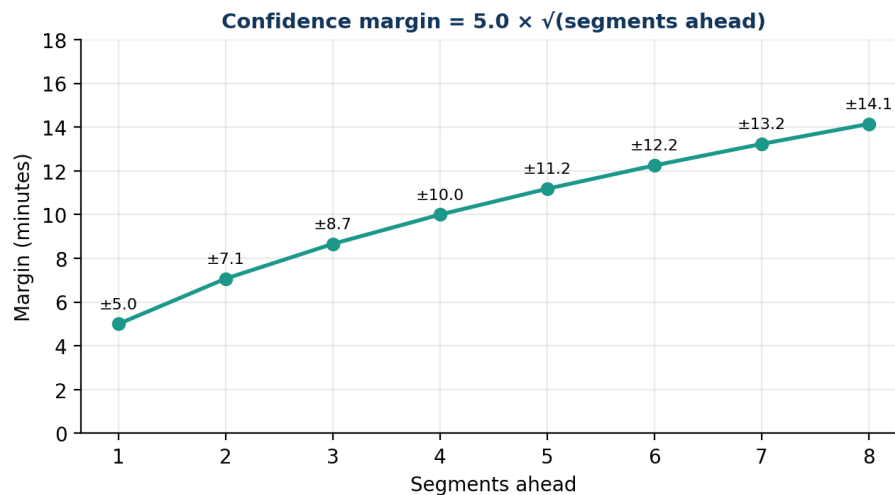


Figure 8. How the heuristic margin widens with the number of segments ahead.

Important caveat

This interval is an uncalibrated MVP heuristic. It is not a statistically validated quantile or conformal prediction interval, and should be described that way to any audience. As a concrete illustration, the margin for five segments ahead is about 11 minutes, while the measured ML MAE at five stations ahead is 66.34 minutes: at long horizons the heuristic band is far narrower than the typical error, which is exactly why calibrated intervals are a priority (Section 19).

10.5 Train and test protocol

- Splitting is done at the **journey level**, not the row level, to prevent leakage between checkpoints of the same trip.
- 200 journeys were shuffled with a fixed seed of 42: 160 for training (4,915 observations) and 40 for testing (1,210 single-segment observations).
- Train and test journey sets are disjoint, and this is verified in the evaluation audit.

Journey-level train / test split (seed 42)

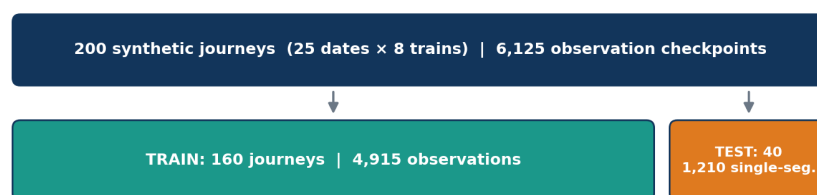


Figure 9. Journey-level split used for training and evaluation.

Splitting by journey matters because consecutive checkpoints of one trip are highly correlated. If rows from the same journey appeared in both training and test data, the model could effectively memorise the trip and the

reported error would be optimistic. Keeping every journey wholly on one side removes that path to leakage.

11. Model Evaluation

The baseline and the ML model were benchmarked on the same 40 held-out journeys, producing 2,430 evaluation observations across three horizons and two operating regimes.

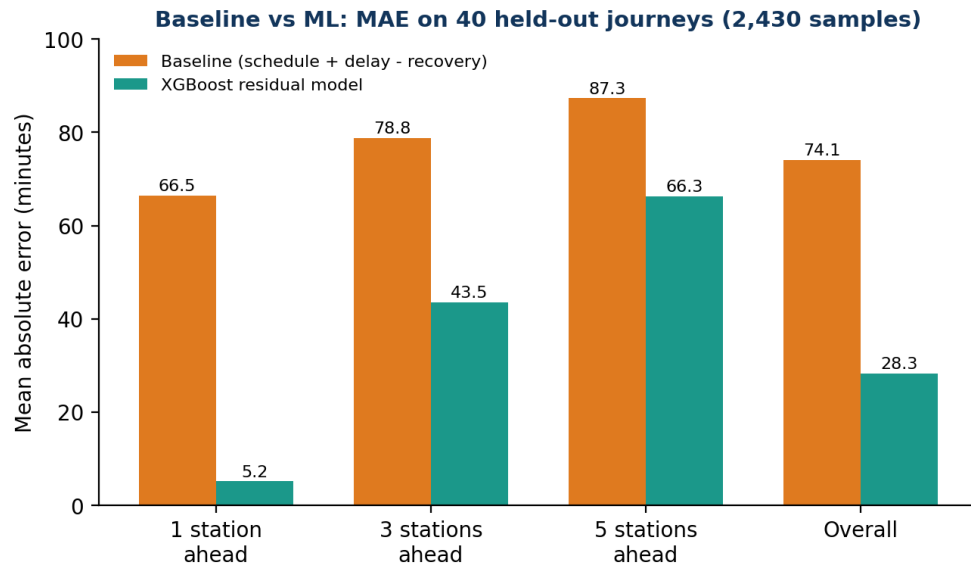


Figure 10. Baseline versus ML: MAE on 40 held-out journeys (2,430 samples).

11.1 By horizon

Table 24. Error by forecasting horizon (minutes).

Horizon	N	Base MAE	Base RMSE	ML MAE	ML RMSE	Gain
1 station	1,210	66.47	96.21	5.17	8.36	+92.2%
3 stations	810	78.83	104.43	43.53	63.67	+44.8%
5 stations	410	87.32	113.19	66.34	89.92	+24.0%

11.2 By operating regime

Table 25. Error by operating regime (minutes).

Regime	N	Base MAE	Base RMSE	ML MAE	ML RMSE	Gain
No disruption	1,585	71.68	99.74	26.93	48.24	+62.4%
With disruption	845	78.65	106.13	30.79	53.86	+60.9%

11.3 Horizon by disruption

Table 26. Error by horizon and regime (minutes).

Horizon	Regime	N	Base MAE	ML MAE	Gain
1 station	No disruption	790	64.71	4.17	+93.5%
1 station	With disruption	420	69.78	7.04	+89.9%
3 stations	No disruption	525	75.91	41.94	+44.8%
3 stations	With disruption	285	84.20	46.46	+44.8%
5 stations	No disruption	270	83.88	64.37	+23.3%
5 stations	With disruption	140	93.96	70.13	+25.4%

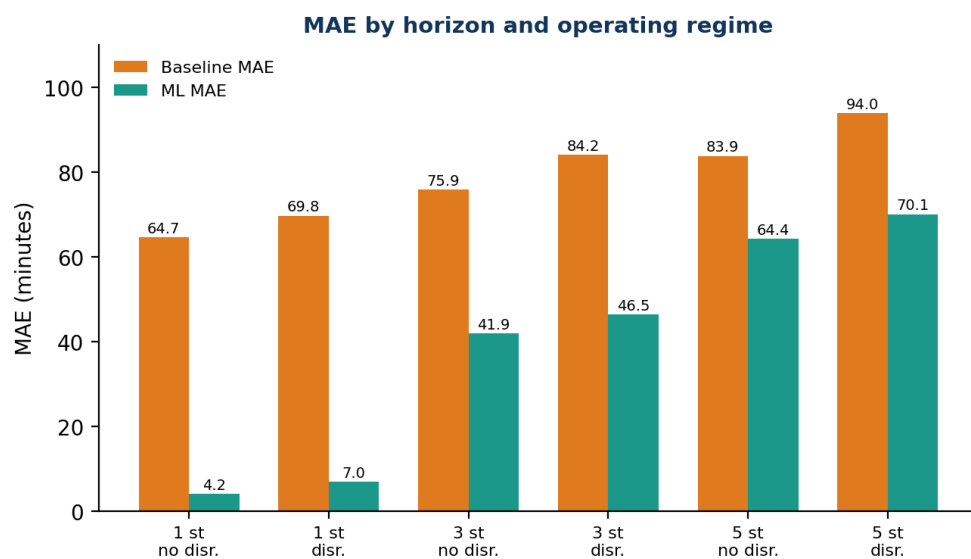


Figure 11. MAE by horizon and operating regime.

11.4 Progression from the first to the final model

Table 27. MAE progression (minutes). The first ML model lost at longer horizons; the residual formulation fixed it.

Category	Baseline	First ML	Final ML	Result
1 station ahead	66.47	5.27	5.17	ML wins throughout
3 stations ahead	78.83	89.11	43.53	Baseline won at first; ML wins after fix
5 stations ahead	87.32	311.58	66.34	Baseline won at first; ML wins after fix
Overall	74.11	84.90	28.27	ML +61.9%

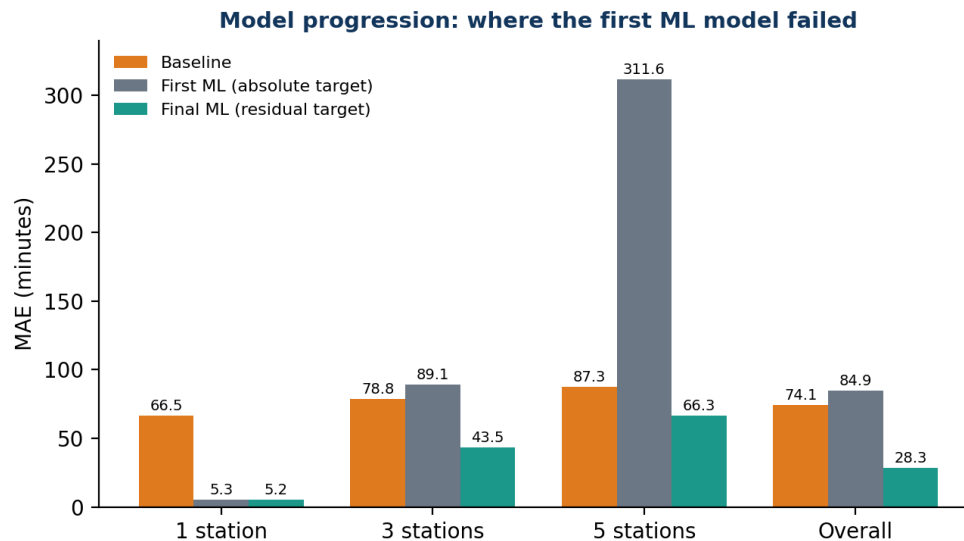


Figure 12. Baseline, first ML model and final ML model compared.

11.5 Interpreting the numbers

- **Short horizon is a large win.** With the next station close, the model captures recovery and restriction effects the baseline cannot.
- **Error grows with horizon, as it should.** Uncertainty accumulates over more segments; the gain narrows to +24% at five stations.
- **Disruption does not break the model.** MAE rises modestly under disruption but the gain over baseline stays near 60%.
- **Baseline error looks high.** The baseline MAE of 66 to 87 minutes reflects the evaluation design and synthetic data; treat it as relative, not as a statement about real IR accuracy.

11.6 Forensic audit

The evaluation report documents an eight-point audit covering feature quality, target construction, simulator realism, train/test leakage, feature distributions, event generation, baseline definition and hyperparameters. The key finding was the target-construction issue described in Section 10.2; leakage was ruled out.

Note on earlier figures

The single-segment model report cites 5.26 min MAE versus a 28.47 min naive baseline (an 81.5% reduction) on the 1,210 single-segment test samples. That is a different benchmark from the multi-horizon table above, and the two should not be mixed.

12. API Reference

The API is served by FastAPI. Interactive OpenAPI documentation is available at </docs> when the server is running. All front-end views use these same endpoints.

Table 28. API endpoints.

Method	Endpoint	Purpose
GET	/health	Health check; returns status, service name and version.
GET	/system/data-source	Configured and effective data-source mode, masked key status, cache statistics.
GET	/trains	All trains with running state and next-station ETAs. Optional <code>data_source</code> query parameter.
GET	/train/{train_id}	Position, delay and full station-wise ETA list (baseline and ML) for one train.
GET	/train/{train_id}/eta/{station_code}	ETA and confidence band for one upcoming station.
GET	/station/{station_code}/arrivals	Arrivals board sorted by predicted ETA. Accepts <code>window_hours</code> .
GET	/live/train/{train_number}	On-demand live lookup through the RailRadar adapter, with ETA computed by the same pipeline.
POST	/simulate/event	Inject a disruption (signal halt, congestion, speed restriction, unscheduled halt, weather) into a running train.
GET	/model/metrics	Latest evaluation metrics by horizon and by disruption regime.
POST	/demo/reset	Reset the deterministic demo scenario.
GET	/demo/scenario	Current state of the demo scenario.
POST	/demo/action/{action_id}	Apply a predefined demo action.

12.1 Example calls

```
# Start the API
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload

# Fleet status
curl http://localhost:8000/trains

# ETA for train 12302 at Howrah
curl http://localhost:8000/train/12302/eta/HWH

# Arrivals at Kanpur in the next 4 hours
curl "http://localhost:8000/station/CNB/arrivals?window_hours=4"
```

The exact request body for `POST /simulate/event` is defined by its Pydantic schema in `backend/main.py` and is visible in the </docs> page.

12.2 Endpoint groups

Table 29. Endpoint groups and their consumers.

Group	Endpoints	Used by
Health and system	/health, /system/data-source	Navigation health badge; operators checking the data-source mode.
Fleet and train	/trains, /train/{id}, /train/{id}/eta/{station}	Control room, passenger lookup, train details.
Station	/station/{code}/arrivals	Station arrivals board.
Live data	/live/train/{number}	On-demand lookups when a RailRadar key is configured.
Simulation control	/simulate/event	Disruption injection form in the control room.
Evaluation	/model/metrics	Model evaluation view and dashboard metrics.
Demo	/demo/reset, /demo/scenario, /demo/action/{id}	Deterministic demonstration mode.

12.3 What an ETA response carries

Endpoints that return ETAs use the same conceptual content, so that every consumer can present the comparison between the current method and the model.

Table 30. Conceptual content of an ETA response (see /docs for exact field names).

Content	Meaning
Predicted ETA	The ML forecast for the station, from the chained segment predictions.
Baseline ETA	Scheduled arrival plus current delay minus recovery buffer, always present.
Confidence bounds	Lower and upper bounds from the horizon-widening heuristic.
Segment breakdown	Per-segment predictions that were summed to produce the ETA.
Provenance	For live lookups: source and cache flags (cached_at, age_seconds, is_cached, is_stale).

12.4 Design notes

- **One source of truth.** The dashboard, passenger view and station board never compute forecasts themselves; they only present what the API returns.
- **Typed contracts.** Pydantic models define responses, and the frontend mirrors them as TypeScript interfaces.
- **Optional data_source parameter.** Fleet queries can select the data source without changing the code path.
- **Sorted by predicted arrival.** The station endpoint orders arrivals by predicted rather than scheduled time, which is what station staff need.

13. Live Data Integration

A RailRadar adapter lets the engine use real running data for on-demand lookups. It is optional; the simulator remains the default and the fallback.

13.1 Data-source modes

Table 31. Data-source modes.

Mode	Behaviour
SIMULATOR	Default. All telemetry comes from the synthetic engine.
LIVE_API	Queries RailRadar per request. If no API key is configured, the effective mode silently remains SIMULATOR.

13.2 Safeguards

- **Secret handling.** The **RAILRADAR_API_KEY** is read from the environment only, never logged, and masked in status responses.
- **No continuous polling.** Queries happen on demand only.
- **Caching.** A thread-safe in-process cache with a default 60-second TTL (configurable) avoids repeated upstream calls, with cooldown after HTTP 429 and stale-on-error serving. Responses carry `cached_at`, `age_seconds`, `is_cached` and `is_stale` flags.
- **Error mapping.** Timeouts, 401, 404, 429, 503 and malformed responses each map to a typed error and a controlled fallback rather than a crash.

13.3 Live flow

Train number, then RailRadar client, then normalised TrainRunningState, then FeatureBuilder, then baseline and ML ETA services. Because the live path reuses the same feature and prediction code as the simulator, no separate model is needed. The figure below shows the decision flow, including the fallback.

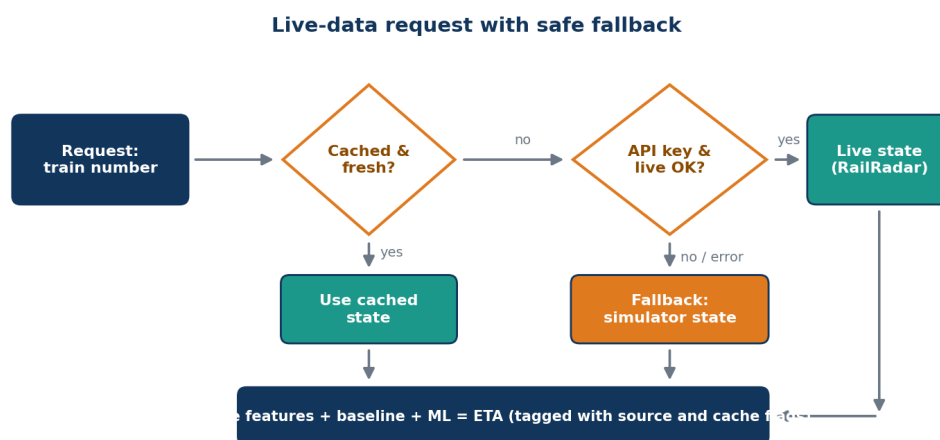


Figure 13. Live-data request with cache and simulator fallback.

13.4 Cache behaviour in practice

Table 32. Cache behaviour.

Scenario	Outcome	Benefit
Repeated lookup within 60 seconds	Served from cache; flags show <code>is_cached</code> .	Protects the third-party quota.
Upstream returns HTTP 429	Cooldown begins; stale data may be served.	Avoids hammering a rate-limited service.
Upstream error while stale data exists	Stale-on-error serving.	Users still receive an answer with <code>is_stale</code> set.
No key configured	Simulator remains the effective mode.	The system works out of the box.

13.5 Adding another data source

The integration boundary is deliberately narrow. To connect a different feed, such as NTES, Control Office Application data or GPS-derived positions, an adapter needs only to emit a `TrainRunningState` for each train. The feature builder, predictor, API and user interface are unchanged. The steps are:

- 1 Implement a client for the new feed with explicit timeouts and typed errors.
- 2 Map the feed's fields onto the normalised running-state schema, tagging the source.
- 3 Register the adapter behind the data-source abstraction and add a mode for it.
- 4 Reuse the existing cache and fallback rules for consistency.
- 5 Retrain and re-evaluate the model on real running logs before making accuracy claims.

14. Frontend

The frontend is a Next.js App Router application using React 19, TypeScript and Tailwind CSS. It only presents data; all business and prediction logic stays in the backend.

Table 33. Frontend routes.

Route	Page	Capabilities
<code>/</code>	Control Room Dashboard	Live fleet table (status, speed, station, delay); disruption injection form; updated baseline and ML ETA after each event; model MAE/RMSE by horizon and regime.
<code>/trains/[id]</code>	Train Details	Telemetry overview; multi-station ETA table with scheduled, baseline and ML times and confidence range; segment breakdown; full route timetable.
<code>/passenger</code>	Passenger ETA Lookup	ETA for any train and station; quick buttons for seeded trains; expected arrival, timetable time, delay badge and confidence range.
<code>/station</code>	Station Arrivals Board	Arrivals sorted by ML ETA; hub selector; 2h, 4h, 8h, 24h and All window filter.

The shared typed client lives in `frontend/src/lib/api.ts`, and TypeScript interfaces in `frontend/src/types/api.ts` mirror the backend Pydantic models. A navigation bar shows a live backend health badge.

14.1 Navigation and shell

The application shell carries the product name, a four-tab navigation bar, an Indian Standard Time clock and a live backend health badge. The badge reads FastAPI: ONLINE when the health endpoint responds, which makes it the quickest check that the frontend is talking to the backend. The tabs are Control Room, Passenger Lookup, Station Board and Model Evaluation.

14.2 Design language

- A light interface with a restrained palette: dark text on white cards, with blue used for ML values and green or red for status.
- A monospaced typeface for identifiers, times and metrics, so that columns of numbers align.
- Prediction tiers are labelled consistently: **SCHED** for the fixed timetable, **BASELINE** for the speed heuristic and **ML ETA** for the XGBoost forecast with an uncertainty range.
- Every screen states its data source (for example a Simulator Mode badge), so it is always clear whether numbers are simulated or live.

14.3 Data flow between views

All views call the same endpoints and hold no forecasting logic of their own. After a disruption is injected in the control room, a refresh of the passenger or station view retrieves the recomputed values, which is why a single event visibly updates every screen.

15. User Interface Walkthrough

This section documents the running application using screenshots captured in Simulator Mode. It follows the order of the navigation bar: Control Room, Passenger Lookup, Station Board and Model Evaluation. The header of every screen shows the IST clock and the FastAPI: ONLINE badge, confirming that the frontend is connected to the backend.

Table 34. The four screens.

Screen	Purpose	Main audience
Control Room	Fleet-wide table of running trains with baseline and ML ETAs, plus disruption injection.	Control room staff
Passenger Lookup	Pick a train and a destination stop and see the expected arrival and its window.	Passengers
Station Board	Inbound queue for a chosen station sorted by predicted arrival.	Station operations
Model Evaluation	Baseline versus ML error metrics overall and by horizon.	Reviewers, evaluators

15.1 Control Room

The Control Room is the fleet-wide view. Its subtitle describes it as real-time telemetry, baseline heuristic ETAs and chained XGBoost forecasts. Above the table, a legend explains the three prediction tiers: SCHED (fixed timetable), BASELINE (speed heuristic) and ML ETA (XGBoost forecast plus uncertainty range).

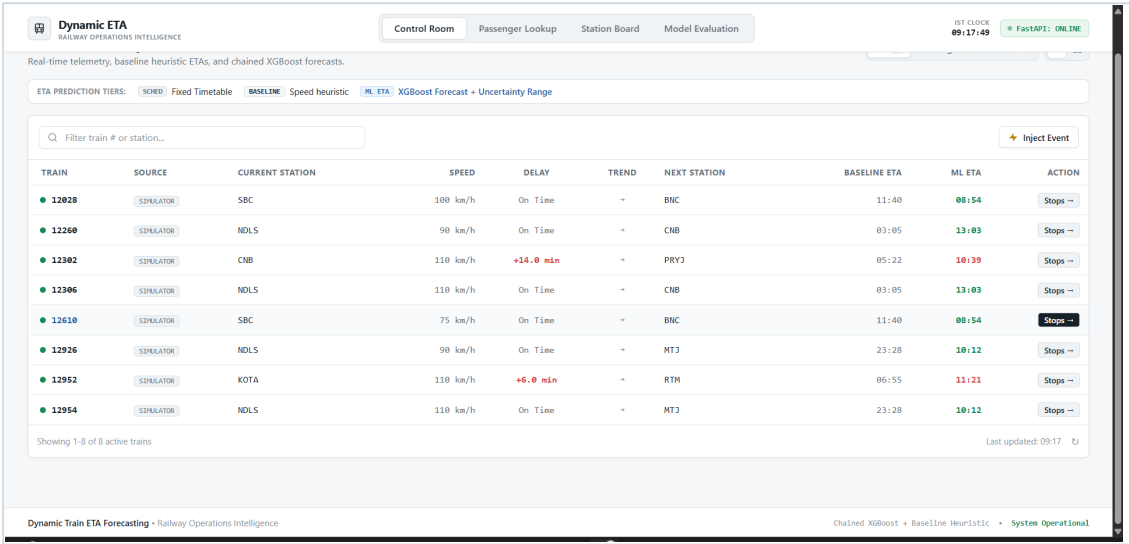


Figure 14. Control Room: fleet table with baseline and ML ETAs for eight active simulated trains.

Each row is one running train. The table lists the train number, data source (all SIMULATOR here), current station, speed, delay, a trend indicator, the next station, the baseline ETA and the ML ETA, and a Stops action that opens the detailed view. A filter box narrows the table by train number or station, and the Inject Event button opens the disruption form. The footer reports Showing 1-8 of 8 active trains and the time of the last update.

Filter train # or station...				
TRAIN	SOURCE	CURRENT STATION	SPEED	DELAY
● 12028	SIMULATOR	SBC	100 km/h	On Time
● 12260	SIMULATOR	NDLS	90 km/h	On Time
● 12302	SIMULATOR	CNB	110 km/h	+14.0 min
● 12306	SIMULATOR	NDLS	110 km/h	On Time
● 12610	SIMULATOR	SBC	75 km/h	On Time
● 12926	SIMULATOR	NDLS	90 km/h	On Time
● 12952	SIMULATOR	KOTA	110 km/h	+6.0 min
● 12954	SIMULATOR	NDLS	110 km/h	On Time

Figure 15. Control Room, left half: train, source, current station, speed and delay columns.

⚡ Inject Event					
ELAY	TREND	NEXT STATION	BASLINE ETA	ML ETA	ACTION
Time	→	BNC	11:40	08:54	Stops →
Time	→	CNB	03:05	13:03	Stops →
.0 min	→	PRYJ	05:22	10:39	Stops →
Time	→	CNB	03:05	13:03	Stops →
Time	→	BNC	11:40	08:54	Stops →
Time	→	MTJ	23:28	10:12	Stops →
0 min	→	RTM	06:55	11:21	Stops →
Time	→	MTJ	23:28	10:12	Stops →

Figure 16. Control Room, right half: trend, next station, baseline ETA, ML ETA and action columns.

Table 35. Values visible in the Control Room capture (times as displayed).

Train	Current station	Speed	Delay	Next station	Baseline ETA	ML ETA
12028	SBC	100 km/h	On Time	BNC	11:40	08:54
12260	NDLS	90 km/h	On Time	CNB	03:05	13:03
12302	CNB	110 km/h	+14.0 min	PRYJ	05:22	10:39
12306	NDLS	110 km/h	On Time	CNB	03:05	13:03
12610	SBC	75 km/h	On Time	BNC	11:40	08:54
12926	NDLS	90 km/h	On Time	MTJ	23:28	10:12
12952	KOTA	110 km/h	+6.0 min	RTM	06:55	11:21

Train	Current station	Speed	Delay	Next station	Baseline ETA	ML ETA
12954	NDLS	110 km/h	On Time	MTJ	23:28	10:12

- **Colour coding.** Delayed trains (12302 at +14.0 min and 12952 at +6.0 min) show their delay and their ML ETA in red; on-time trains show the ML ETA in green.
- **Trains on the same corridor share a next station.** 12028 and 12610 are both at SBC heading for BNC, and 12926 and 12954 are both at NDLS heading for MTJ.
- **Speed differs per train.** Speeds range from 75 km/h (12610) to 110 km/h, which is the simulator's stated maximum.

15.2 Passenger Lookup

The Passenger ETA Portal lets a traveller choose a train and a destination stop and see when the train is expected. A Simulator Mode badge, a Sync button and an IST time make the data source and freshness explicit.

Step 1: search or select a train

The first card offers a search box (by train number or name), a drop-down of the eight active trains and quick-select buttons for seeded trains. In the capture, train 12028, the KSR Bengaluru to MGR Chennai Shatabdi, is selected. Its summary shows the RUNNING status and an On Time badge, together with the train category, route number and the SIMULATOR source tag.

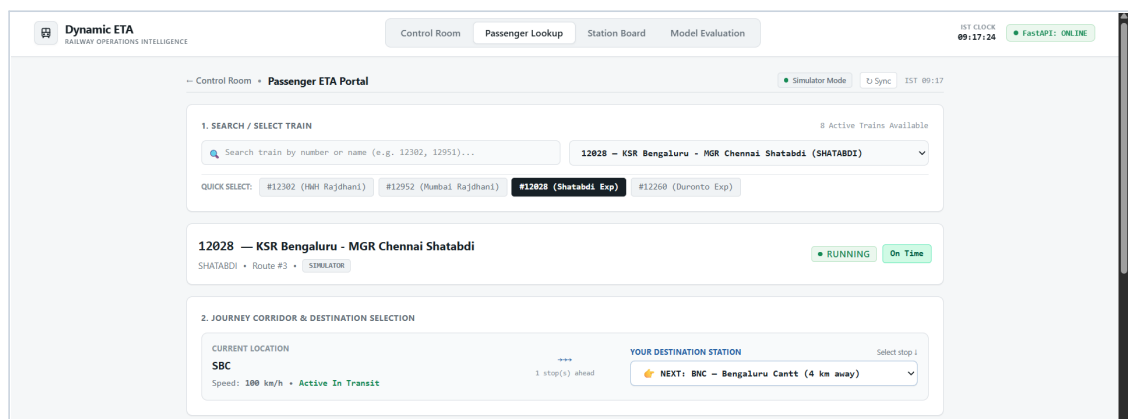


Figure 17. Passenger Lookup, steps 1 and 2: choosing a train and a destination station.

Step 2: choose the destination

The second card, Journey Corridor and Destination Selection, shows the train's current location (SBC), its speed (100 km/h) and status (Active In Transit), and the number of stops ahead. A drop-down lists the upcoming stops; the default is the next stop, here BNC (Bengaluru Cantt), 4 km away.

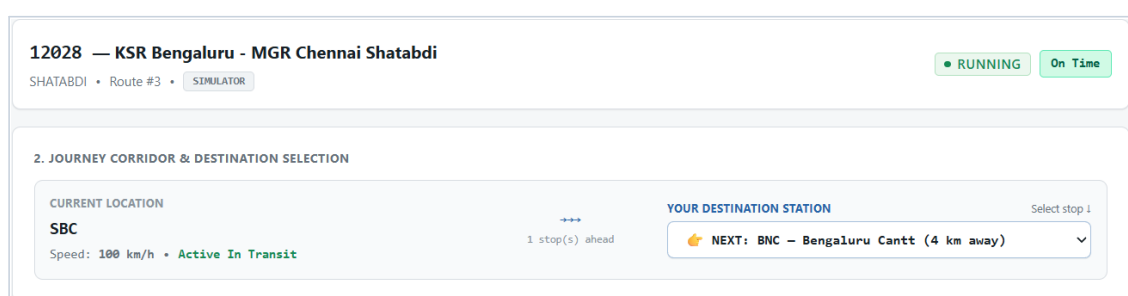


Figure 18. Zoom: current location, stops ahead and the destination selector.

Step 3: read the dynamic ETA forecast

Once a destination is chosen, the ETA forecast card shows the result. It is headed Dynamic ETA Forecast: Arrival at Bengaluru Cantt (BNC), with an ML Chained Forecast tag and the distance to go (4.0 km). Three summary tiles follow: the estimated arrival (ML), the estimated transit remaining and the prediction range.

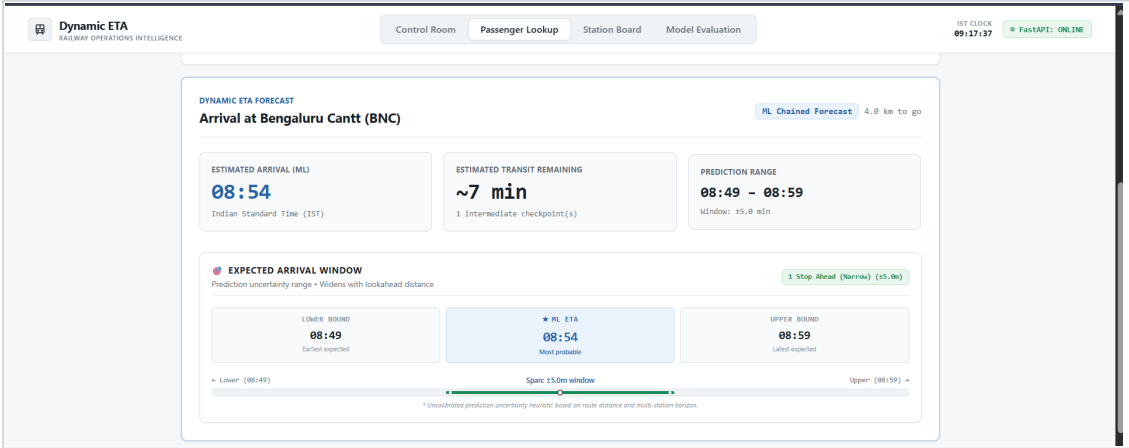


Figure 19. Passenger Lookup, step 3: dynamic ETA forecast with the expected arrival window.

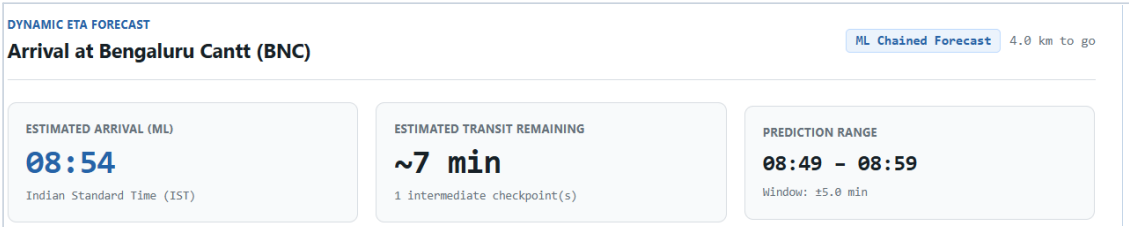


Figure 20. Zoom: estimated arrival 08:54, about 7 minutes of transit remaining and a range of 08:49 to 08:59.

Table 36. Reading the ETA forecast card.

Element	Value in capture	Meaning
Estimated arrival (ML)	08:54 IST	Most probable arrival time from the chained forecast.
Estimated transit remaining	~7 min; 1 intermediate checkpoint	Time still to run, with the number of checkpoints between now and the destination.
Prediction range	08:49 to 08:59; window +/- 5.0 min	Lower and upper bound from the uncertainty heuristic.
Horizon badge	1 Stop Ahead (Narrow), +/- 5.0 m	Margin category; matches 5.0 x sqrt(1) for one segment.
Footnote	Uncalibrated uncertainty heuristic	Labels the band honestly as a heuristic, in line with Section 10.4.

Consistent with the methodology

The +/- 5.0 minute window for a one-stop-ahead forecast is exactly the first value of the heuristic $5.0 \times \sqrt{\text{segments ahead}}$. For a destination several stops ahead the same card would show a wider window, and the footnote on the screen states that the band is an uncalibrated heuristic.

15.3 Station Board

The Digital Station Arrivals Board shows inbound trains for one station, sorted by predicted arrival. The controls at the top take a station code and a time window, with a Load Board button and a row of major-junction shortcuts. Two further buttons, Passenger Portal and Refresh Board, sit at the top right.

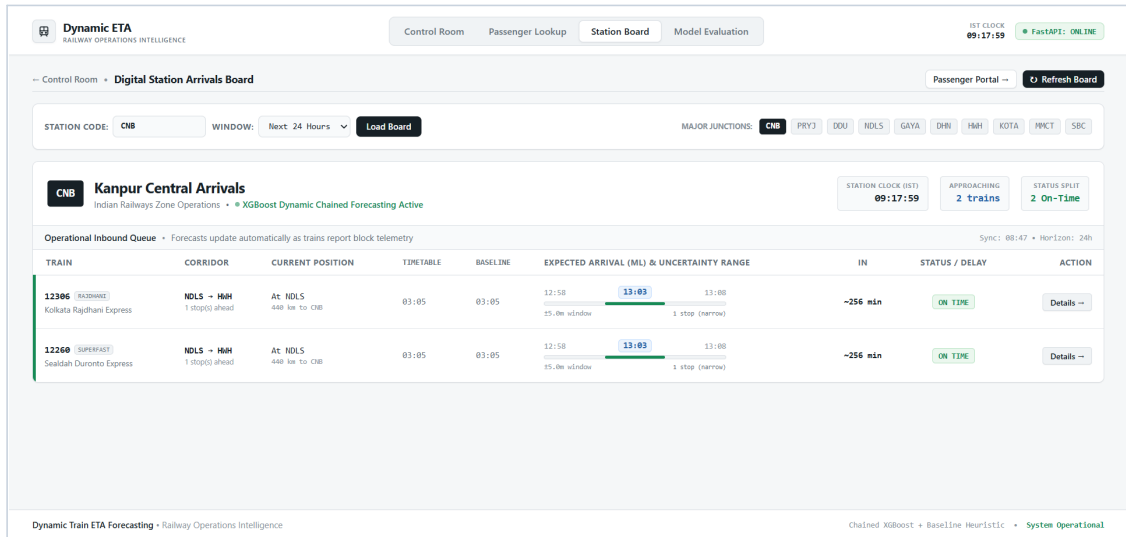


Figure 21. Station Board for CNB (Kanpur Central): two inbound trains with ML ETAs and uncertainty ranges.

In the capture the station is CNB with a Next 24 Hours window. The station header shows the station clock, the number of approaching trains (2) and a status split (2 On-Time). The Operational Inbound Queue then lists each train with its corridor, current position, timetable time, baseline time, and a bar showing the ML expected arrival with its uncertainty range, followed by the minutes remaining and a status badge.

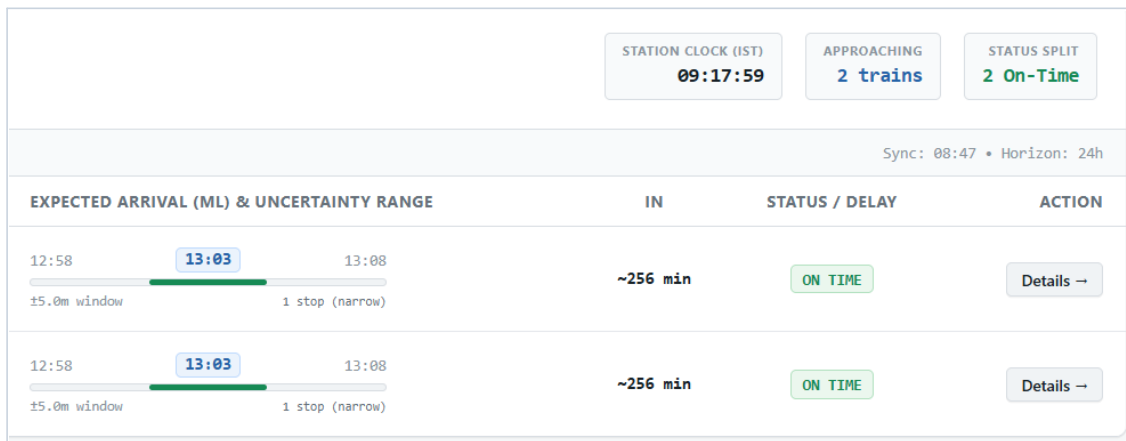


Figure 22. Zoom: expected arrival and uncertainty range, minutes remaining, status and Details action.

Table 37. Values visible in the Station Board capture.

Train	Corridor	Position	Timetable	Baseline	ML ETA (range)	In	Status
12306 Kolkata Rajdhani Express	NDLS to HWH	At NDLS, 440 km to CNB	03:05	03:05	13:03 (12:58 to 13:08)	~256 min	ON TIME
12260 Sealdah Durgam Express	NDLS to HWH	At NDLS, 440 km to CNB	03:05	03:05	13:03 (12:58 to 13:08)	~256 min	ON TIME

Both trains are at NDLS, one stop ahead of CNB in the sequence shown, and the interface labels the band as 1 stop (narrow), with a window of +/- 5.0 minutes. The interface also states the sync time (08:47) and the horizon (24h) of the queue, and notes that forecasts update automatically as trains report block telemetry.

15.4 Model Evaluation

The Model Evaluation and Performance Benchmarks screen presents the empirical comparison between the XGBoost dynamic chained forecasts and the baseline speed heuristic, computed from the metrics endpoint. A Refresh button reloads the numbers, and a timestamp shows when they were last loaded.

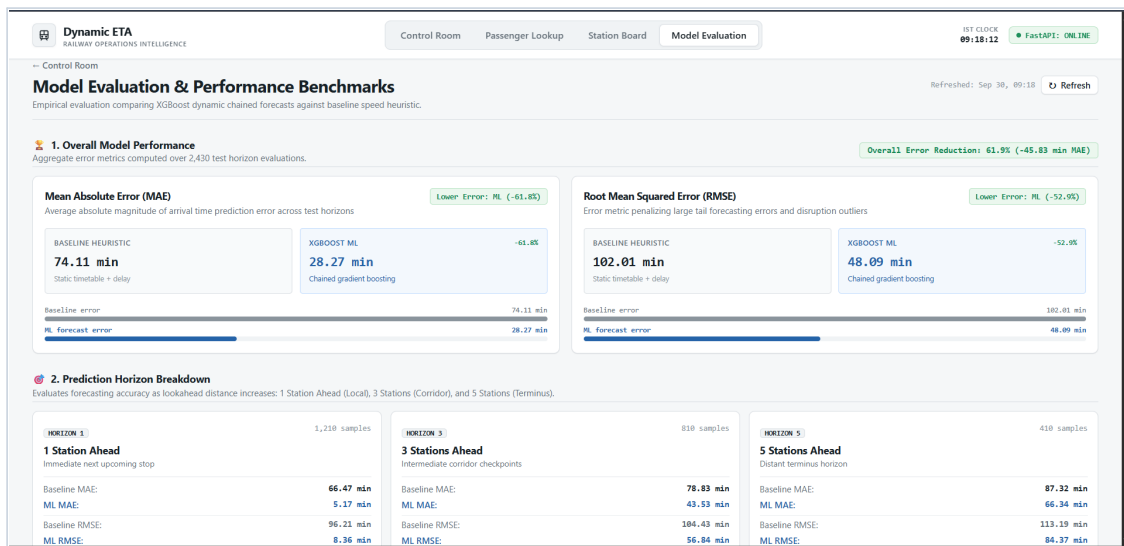


Figure 23. Model Evaluation: overall MAE and RMSE, followed by the prediction horizon breakdown.

Section 1, Overall Model Performance, aggregates error over 2,430 test horizon evaluations and highlights an overall error reduction of 61.9% (a 45.83 minute reduction in MAE). It places baseline and ML values side by side for MAE and RMSE, with proportional bars. Section 2, Prediction Horizon Breakdown, repeats the comparison for one, three and five stations ahead, with the number of samples in each.

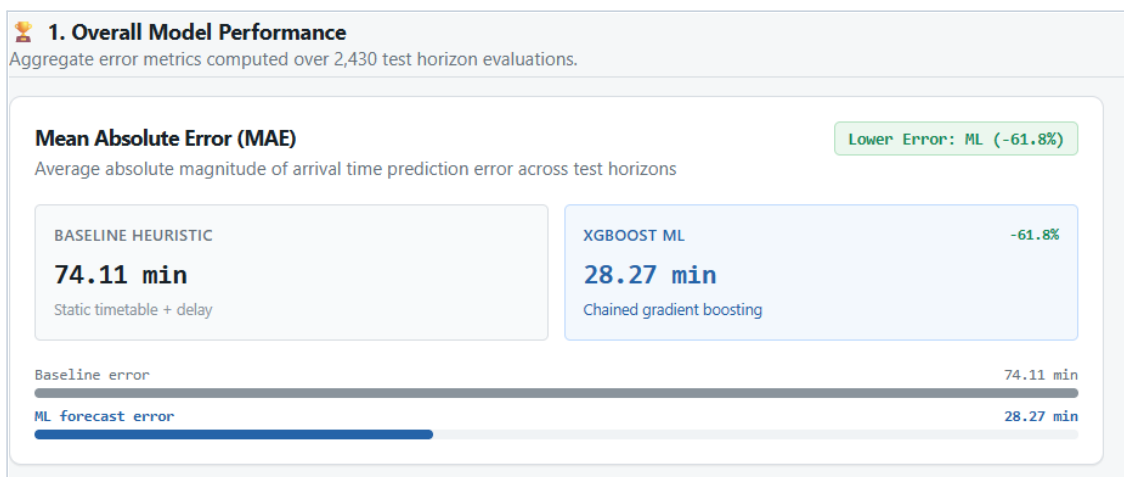


Figure 24. Zoom: Mean Absolute Error card with baseline 74.11 min against XGBoost 28.27 min.

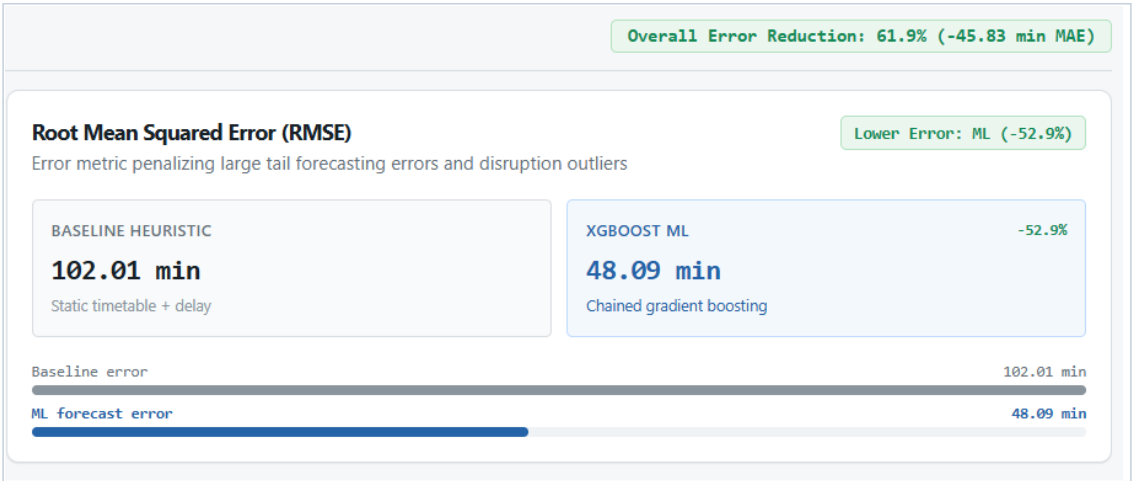


Figure 25. Zoom: Root Mean Squared Error card with baseline 102.01 min against XGBoost 48.09 min.

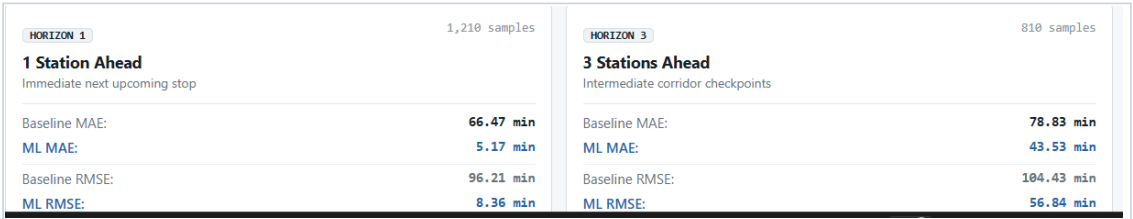


Figure 26. Zoom: horizon breakdown cards for one and three stations ahead.

Table 38. Values visible on the Model Evaluation screen.

Metric	Baseline	ML	Reduction shown
Overall MAE	74.11 min	28.27 min	-61.8% (card); 61.9% (banner)
Overall RMSE	102.01 min	48.09 min	-52.9%
Horizon 1: MAE / RMSE (1,210 samples)	66.47 / 96.21	5.17 / 8.36	n/a
Horizon 3: MAE / RMSE (810 samples)	78.83 / 104.43	43.53 / 56.84	n/a
Horizon 5: MAE / RMSE (410 samples)	87.32 / 113.19	66.34 / 84.37	n/a

15.5 Observations from the captures

Comparing the screenshots with the rest of this report surfaces a few points that are worth knowing before a live demonstration. None of them changes the method or the headline results in Section 11; they are recorded so that the numbers on screen are not misread.

Table 39. Points to check before a live demonstration.

Observation	Detail	Suggested action
Rounding on the Model Evaluation screen	The MAE card shows -61.8% while the banner shows 61.9%. The exact ratio is (74.11 - 28.27) / 74.11 = 61.86%, and the absolute reduction is 45.84 min against 45.83 shown.	Round consistently, for example to 61.9% everywhere.
ML RMSE at horizons 3 and 5	The screen shows 56.84 and 84.37 min, whereas the evaluation tables in Section 11 report 63.67 and 89.92 min. Baseline RMSE and all MAE values agree.	Confirm which metrics file the screen reads and regenerate so both match.

Observation	Detail	Suggested action
Simulator clock versus IST clock	The header shows IST 09:17, but the ETAs imply a simulator time of about 08:47: 08:54 arrival with about 7 min to go, and the Station Board sync time is 08:47.	State that ETAs are in simulator time during a demo.
Timetable and baseline versus ML times	For trains labelled On Time the Timetable and Baseline times differ from the ML ETA by many hours (for example 12926: 23:28 against 10:12), so they appear to sit on different time bases in the simulator.	Check the time base before comparing columns side by side.
Baseline equals timetable on the board	Both Station Board rows show baseline 03:05 equal to the timetable, while the formula in Section 10.1 would subtract a recovery buffer of about 11 min over 440 km.	Verify how the board renders the baseline.
Junction shortcuts	The Station Board shows CNB, PRYJ, DDU, NDLS, GAYA, DHN, HWH, KOTA, MMCT and SBC, while the frontend table in Section 14 lists NDLS, CNB, PRYJ, DDU, KOTA, MAS, HWH and MMCT.	Update whichever list is out of date.

These observations are drawn only from the five supplied screenshots and the original documentation. They are flagged for verification rather than stated as defects.

16. Demonstration Scenario

A deterministic demo mode makes the live demonstration fully reproducible (random seed 42).

Table 40. Demo starting state.

Setting	Value
Train	12302 Howrah Rajdhani Express
Position	Passing Kanpur Central (CNB), heading to Prayagraj Jn (PRYJ), 195 km to go
Segment progress / speed	15% / 110 km/h
Initial delay / status	2 minutes / RUNNING

Table 41. Predefined demo actions.

Demo action	Effect
Signal halt	+15 min, speed 0 km/h, status HALTED
Congestion	+10 min, speed 49.5 km/h, status RUNNING
Speed restriction	+8 min, speed 30 km/h, status RUNNING

16.1 What happens when you inject an event

Live demo: inject a disruption, watch every screen update



Figure 27. Demo event loop: one injection updates every screen.

When an event is injected, the simulator updates delay, speed and status for the train; features and ETAs are recomputed on the next request; and the dashboard, passenger view and station board show the revised values on refresh.

16.2 Suggested five-minute script

- 1 **Open (30 s):** static schedule plus buffer versus real-time conditions.
- 2 **Architecture (30 s):** show the architecture diagram (Section 5) and point at the live-data plug-in.
- 3 **Live demo (2 min):** open the dashboard, select 12302, compare baseline and ML ETA, inject a signal halt, and watch the dashboard, passenger view and station board update.
- 4 **Results (30 s):** show the MAE table, emphasising the disruption rows.
- 5 **Close (30 s):** scalability argument and what real IR integration would need.

16.3 Speaking points

- State up front that the data are synthetic and the value is the method and the relative gain.
- Point at the baseline column and the ML column side by side: the comparison is always visible.
- After injecting the signal halt, note that the ETA moves without any restart, on every screen.
- When showing the confidence band, describe it as a transparent heuristic that widens with horizon.
- Use the disruption rows of the results table to show that the gain holds near 60% under disruption.

16.4 Rehearsal notes

Reset the demo scenario before each rehearsal so the starting state is identical, and keep a recorded backup demonstration in case the live demo fails (see the pre-demo checklist in Section 24). Because the simulator clock and the IST header clock can differ (Section 15.5), explain that times shown for the forecast are simulator times.

17. Testing and Quality Assurance

The automated suite contains over 150 test functions across 16 modules, run with pytest.

Table 42. Test coverage by area.

Area	Test module(s)
API contract	test_api.py (largest module)
Baseline and multi-station ETA	test_baseline_eta.py, test_multi_station_eta.py
Features and dataset	test_features.py, test_dataset_generator.py
Model and evaluation	test_predictor.py, test_evaluation.py
Simulator and database	test_simulator.py, test_database.py
Live data path	test_railway_api_client.py, test_data_source.py, test_cache.py, test_live_api_integration.py, test_live_feature_pipeline.py, test_live_train_lookup.py
Demo mode	test_demo_scenario.py

```
python -m pytest
python verify_day1.py # Day 1 end-to-end verification script
```

Earlier project reports record 62 of 62 tests passing at the end of Day 2; the suite has grown since. Rerun it before any release to confirm the current status.

17.1 Testing strategy

- **Contract tests** exercise the API endpoints and check that response shapes match the schemas the frontend expects.
- **Unit tests** cover the baseline arithmetic, feature construction, the simulator kinematics and the repository helpers.
- **Model and evaluation tests** check the predictor and the evaluation module, including the journey-level split.
- **Live-path tests** cover the client, the cache, the data-source abstraction and the end-to-end live feature pipeline, so error mapping and fallback are verified without depending on the network.
- **Demo test** confirms that the deterministic scenario produces the same starting state every time.

17.2 Quality practices reflected in the design

- Journey-level splitting and an explicit train/test disjointness check in the evaluation audit.
- A forensic, eight-point audit of the evaluation, with the early failure documented rather than hidden.
- Typed contracts end to end (Pydantic on the server, TypeScript on the client).
- Secrets handled through the environment only, and masked in status responses.

17.3 What is not yet covered

Formal load testing, authentication and observability are out of scope for this build (Section 4.2) and should be added before any pilot. Model accuracy on real data has likewise not been tested, because only synthetic data were available.

18. Installation and Operation

18.1 Prerequisites

- Python 3 with pip
- Node.js and npm (frontend)

18.2 Backend

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

18.3 Frontend

```
cd frontend
npm install
npm run dev
# open http://localhost:3000
```

18.4 Model artefacts

Serialised model files live under `data/models` (`eta_xgboost_model.json` and `eta_xgboost_metadata.json`), and processed datasets and metrics under `data/processed`. These folders are tracked with placeholder files, so regenerate the dataset, retrain and re-evaluate with the modules in `backend/ml/` if the artefacts are absent.

18.5 Configuration

Table 43. Configuration variables.

Variable	Default	Purpose
HOST / PORT	127.0.0.1 / 8000	API bind address.
ENVIRONMENT / LOG_LEVEL	development / info	Runtime mode and logging.
DATABASE_URL	sqlite:///./data/train_eta.db	SQLite database location.
SIMULATION_TICK_SECONDS	2.0	Real seconds between simulator ticks.
SIMULATED_MINUTES_PER_TICK	2.0	Simulated minutes advanced per tick.
RAW/PROCESSED/MODELS_DIR	./data/...	Storage paths.
RAILRADAR_API_KEY	(empty)	Optional; enables live mode. Keep out of version control.
RAILRADAR_BASE_URL	https://api.railradar.io	Live API base URL.
RAILRADAR_CACHE_TTL_SECONDS	60	Live-data cache lifetime.

18.6 Operating the system

- 1 Start the backend and confirm that the health endpoint responds.
- 2 Start the frontend and confirm that the navigation bar shows FastAPI: ONLINE.
- 3 Open the Control Room and confirm that trains are listed and updating.
- 4 Optionally set RAILRADAR_API_KEY to enable live lookups; otherwise the system remains in simulator mode.
- 5 After any model change, re-run the evaluation and refresh the Model Evaluation screen.

18.7 Speeding up or slowing down the simulation

Two variables control how quickly the simulated world moves: the real seconds between ticks and the simulated minutes advanced per tick. With the defaults (2.0 and 2.0), one simulated minute passes per real second, which is the 60x rate mentioned in the simulator description. Lengthening the tick interval slows the world down for a careful walkthrough; shortening it speeds up the evolution of trains along their routes.

19. Limitations and Risks

Table 44. Limitations, impact and mitigation.

Limitation	Impact	Mitigation / path forward
Synthetic training and test data	Reported MAE does not represent real IR performance.	Retrain and re-evaluate on real running logs; the source-agnostic design supports this.
Historical features use defaults	Section-level history is not yet informative.	Supply real per-section, per-weekday statistics via SectionStatsProvider.
Confidence band is heuristic	Not a calibrated probabilistic interval.	Adopt quantile regression or conformal prediction.
Error grows with horizon	5-station MAE is 66 min in the simulator.	Add network-level features such as preceding-train state; consider sequence models.
Only three corridors, eight trains	Limited route diversity.	Add corridors and train types.
No authentication or load testing	Not production hardened.	Add auth, observability and performance testing before any pilot.
Live API depends on a third party	Quota, latency and availability risk.	Caching, cooldown and simulator fallback already in place.

19.1 Discussion

Data realism. The most important limitation is that every accuracy figure comes from a simulator. The simulator was audited for realism as part of the evaluation, and leakage was ruled out, but the absolute numbers cannot be transferred to real operations. What transfers is the formulation and the demonstration that it beats the baseline under controlled conditions.

Long-horizon error. At five stations ahead the model still wins, but by a smaller margin (+24.0%) and with a large absolute error (66.34 min). This is expected, since uncertainty compounds across segments, and it motivates network-level features and cross-train effects.

Uncertainty communication. The band is transparent but uncalibrated. Presenting it as a validated interval would overstate what is known, which is why the interface footnote and this report both label it as a heuristic.

Operational maturity. Authentication, observability and load testing are not part of this MVP. Any pilot would need them, along with monitoring of live forecast error against realised arrivals.

20. Scalability and Extensibility

- **Independent per-train computation.** Features for each train depend only on that train's state and the timetable, so work shards naturally, for example by railway zone.
- **Small, cheap model.** One gradient-boosted model applied per segment, not per train, is inexpensive to serve.
- **Decoupled retraining.** Batch retraining can run separately from serving, so prediction latency is unaffected as the model improves.
- **Zone-aligned deployment.** The zonal structure of Indian Railways maps onto zone-sharded instances of the same pipeline.
- **Clean integration boundary.** Replacing the simulator with NTES, COA or GPS-derived feeds only requires an adapter that emits TrainRunningState.

20.1 Roadmap

Table 45. Roadmap.

Horizon	Item
Near term	Ingest a real public delay dataset alongside the simulator, clearly labelled; calibrate confidence intervals; add online error-correction so ETAs tighten as a train nears a station.
Mid term	Cross-train effects (preceding-train delay propagation); per-zone models; scheduled retraining; observability and authentication.
Longer term	Sequence-model comparison; downstream integration with platform and crew planning; pilot on a live zone.

20.2 A path from prototype to pilot

- 1 Connect one real feed through an adapter, clearly labelled, and run it beside the simulator.
- 2 Retrain and re-evaluate on real running logs; replace synthetic historical defaults with real section statistics.
- 3 Calibrate the confidence band, then re-label it according to what has been validated.
- 4 Add authentication, observability and load testing.
- 5 Run a pilot on one railway zone, tracking forecast error against realised arrivals.

21. Innovation Highlights

This section separates what is already built and demonstrated from ideas that are proposed as next steps. The first part is grounded in the code and reports; the second part is clearly labelled as proposals.

21.1 Innovations delivered in this build

Table 46. Innovations delivered.

Innovation	What it is	Why it matters
Schedule-residual target	The model predicts the gap between actual and scheduled remaining time, not raw minutes.	Fixed tree models failing to extrapolate. Turned losses at 3 and 5 stations into wins of +44.8% and +24.0%.
Baseline always beside ML	The current-practice baseline is computed and shown next to the ML ETA for every station.	Makes the improvement visible, and guarantees an answer even if the ML model is unavailable.
Source-agnostic train state	Simulator and live API both emit one TrainRunningState schema.	Swapping in real feeds needs an adapter only, with no change to features, model, API or UI.
Segment chaining with widening uncertainty	Per-segment predictions are chained and the confidence margin grows as $5 \times \sqrt{\text{segments ahead}}$.	One small model forecasts every downstream station and the UI shows a range, not false precision.
Live disruption injection	Signal halts, congestion, restrictions, unscheduled halts and weather can be injected on demand.	Shows the ETA reacting in seconds on every screen, which a static timetable cannot do.
Deterministic demo mode	A fixed seed and preset scenario (train 12302 near Kanpur) give identical results every run.	Removes demo-day randomness and makes judging reproducible.
Safe live-data integration	Cached with a 60 s default TTL, cooldown after HTTP 429, stale-on-error serving, masked API key, no continuous polling.	Protects quota and secrets, and the system degrades gracefully instead of failing.
Honest, leakage-safe evaluation	Journey-level train/test split, results by horizon and by disruption regime, and the early failures documented.	Builds trust: the report shows where ML first lost and exactly how it was fixed.

Why the residual idea is the standout

A direct regression on remaining minutes failed at long horizons in this project. Anchoring on the timetable and learning only the operational deviation is simple, cheap and easy to reuse: it keeps the target in a stable range (about -90 to +60 minutes) no matter how far ahead the forecast looks.

21.2 Proposed innovations (not yet built)

These are suggestions that fit the existing architecture. None of them is implemented, and their benefit has not been measured.

Table 47. Proposed innovations (not built).

Proposal	Idea	How it fits
Cross-train cascade	Feed the delay and position of the preceding train on the same section into the features.	Adds a network-level feature; targets the long-horizon error that grows to 66 min at five stations.
Calibrated intervals	Replace the sqrt heuristic with quantile regression or conformal prediction.	Turns the confidence band into a statistically valid range, once real data exists.
Explain the delay	Show passengers the main reason for a revised ETA, for example congestion ahead or an active restriction, using feature attributions such as SHAP.	Uses the existing 11 features, so explanations are readable.
Connection-risk alerts	Warn when a predicted delay puts a connecting train or a platform slot at risk.	Downstream use of the same API, described in the PRD as a stretch idea.
Online error correction	Nudge each train's forecast with an exponentially weighted recent error, so ETAs tighten near a station without retraining.	Listed as a stretch goal in the PRD; needs no change to the API.
Zone-sharded deployment	Run one pipeline instance per railway zone.	Per-train features are independent, so work shards naturally.

22. Glossary

Table 48. Glossary.

Term	Definition
ETA	Expected Time of Arrival.
MAE / RMSE	Mean absolute error and root mean squared error, in minutes. RMSE penalises large errors more.
Baseline	Scheduled arrival plus current delay minus scheduled recovery buffer.
Residual target	Actual remaining minutes minus scheduled remaining minutes.
Horizon	How many stations ahead the forecast looks.
TSR	Temporary speed restriction.
Segment chaining	Summing per-segment predictions to forecast a distant station.
NTES / COA	National Train Enquiry System / Control Office Application, real IR systems for a future integration.
PRD	Product Requirements Document (in docs/).
TrainRunningState	The normalised schema describing a train's running state, emitted by both the simulator and the live adapter.
Simulator mode	Data-source mode in which all telemetry comes from the synthetic engine.
TTL	Time to live: how long a cached live response is considered fresh.
Leakage	Information from test data influencing training, which would make reported error optimistic.

23. Source Documents

- [docs/PRD_Dynamic_ETA_Forecasting.pdf](#): product requirements.
- [docs/day2_dataset_report.md](#): dataset quality audit.
- [docs/model_training.md](#): XGBoost training report.
- [docs/model_evaluation.md](#): baseline versus ML evaluation and forensic audit.
- [docs/frontend_guide.md](#): frontend architecture and usage.

The user-interface walkthrough in Section 15 is based on five screenshots of the running application captured on 30 September 2026 in Simulator Mode.

24. Conclusion

The Dynamic ETA Forecasting Engine shows that a small, well-formulated gradient-boosted model can outperform the static schedule-plus-buffer approach across every forecasting horizon and operating regime tested, while staying cheap to serve and simple to operate. Overall error fell from 74.11 to 28.27 minutes on held-out simulated journeys, and the largest gain, 92.2%, came at the short horizon that matters most to passengers and station staff.

Equally important is what the project makes visible. The baseline is always shown next to the ML forecast, the confidence band is clearly labelled as a heuristic, the synthetic origin of the data is stated openly, and the source-agnostic design gives a clear route to real Indian Railways data. That combination of measurable improvement and honest framing is what makes the work credible.

24.1 Key takeaways

- Anchoring the target to the timetable (the residual formulation) was the decisive modelling change, turning a model that lost at long horizons into one that wins everywhere.
- One API and one normalised state object serve the dashboard, passenger view, station board and live-data path.
- Failure handling is built in: baseline always available, live-API errors fall back to the simulator.
- The next milestone is retraining and re-evaluating on real running logs, with calibrated confidence intervals.

24.2 Pre-demo checklist

- 1 Install dependencies, copy `.env.example` to `.env`, and confirm the model files exist in `data/models`.
- 2 Start the backend and confirm the `/health` endpoint responds.
- 3 Start the frontend and confirm the navigation bar shows a healthy backend badge.
- 4 Reset the demo scenario and rehearse the signal-halt injection on train 12302.
- 5 Run `python -m pytest` and record the current pass count.
- 6 Keep a recorded backup demo video in case the live demo fails.

In one line

A schedule-anchored gradient-boosted model cut overall ETA error from 74.11 to 28.27 minutes on held-out simulated journeys, with the baseline always shown alongside it and a clear path to real Indian Railways data.

Appendix A. Worked Calculations

A.1 Improvement percentages

The improvement figures in the results tables follow one formula: $\text{improvement} = (\text{baseline MAE} - \text{ML MAE}) / \text{baseline MAE}$.

Table 49. Recomputing the improvement column.

Horizon	Baseline MAE	ML MAE	Difference	Improvement
1 station	66.47	5.17	61.30	$61.30 / 66.47 = 92.2\%$
3 stations	78.83	43.53	35.30	$35.30 / 78.83 = 44.8\%$
5 stations	87.32	66.34	20.98	$20.98 / 87.32 = 24.0\%$
Overall	74.11	28.27	45.84	$45.84 / 74.11 = 61.9\%$

A.2 Baseline ETA

Baseline ETA = scheduled arrival + current delay - recovery buffer, where the buffer is 2.5 minutes per 100 km. For a target 440 km away the buffer is $2.5 \times 4.4 = 11$ minutes; for a target 195 km away it is 4.875 minutes.

A.3 Confidence margin

Margin = $5.0 \times \sqrt{n}$ minutes for n segments ahead: $n = 1$ gives 5.00; $n = 2$ gives 7.07; $n = 3$ gives 8.66; $n = 4$ gives 10.00; $n = 5$ gives 11.18.

A.4 Residual target

If the timetable says a segment takes 60 minutes and the train actually takes 48 minutes, the residual target is $48 - 60 = -12$ minutes and the model prediction is $60 + (\text{predicted residual})$. If the train takes 75 minutes the residual is +15. Both fall inside the roughly -90 to +60 minute range, whatever the length of the trip.

A.5 Splits

200 journeys (25 dates x 8 trains) were split 160 train and 40 test. The 40 test journeys yield 1,210 single-segment observations; across the three horizons the same journeys yield 2,430 evaluation observations ($1,210 + 810 + 410$).

Appendix B. Troubleshooting Guide

Table 50. Troubleshooting.

Symptom	Likely cause	What to do
Navigation badge is not ONLINE	Backend not running, or wrong host or port.	Start uvicorn on port 8000 and check /health.
No trains listed	Simulator not started, or database not initialised.	Restart the backend; tables are initialised at start-up.
Only baseline ETAs appear	ML model files missing from data/models.	Regenerate the dataset, retrain and re-evaluate with backend/ml/.
Live lookup returns simulator data	No RAILRADAR_API_KEY, or the live API errored.	Set the key in .env; check provenance flags in the response.
Live lookup returns stale data	Upstream error or HTTP 429 cooldown.	Wait for cooldown; check is_stale and age_seconds.
Model Evaluation empty	Metrics file absent from data/processed.	Re-run the evaluation module and refresh.
Numbers differ between report and screen	Screen reads a different metrics run.	See Section 15.5 and regenerate metrics.
Demo not reproducible	Scenario not reset.	POST /demo/reset before each rehearsal.

Appendix C. Frequently Asked Questions

Do the results show how accurate Indian Railways ETAs are?

No. All figures come from a synthetic simulator. They show that the method beats a schedule-plus-delay baseline under identical, controlled conditions.

Why is the baseline error so large?

The baseline MAE of 66 to 87 minutes reflects the evaluation design and synthetic data. It should be used only as a relative comparison.

Why not predict remaining minutes directly?

That was the first attempt. Tree models cannot extrapolate beyond the target range seen in training, so long-horizon predictions plateaued near 310 minutes. The residual target fixes this.

Is the confidence band statistically valid?

No. It is a transparent heuristic, $5.0 \times \sqrt{\text{segments ahead}}$ minutes, and is labelled as uncalibrated in both the interface and this report.

Can real data be used?

Yes, through an adapter that emits TrainRunningState. An optional RailRadar adapter already supports on-demand lookups, with caching and fallback.

References

A. Research papers and books

- [1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [2] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [3] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.
- [4] V. Vovk, A. Gammerman and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.
- [5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017.
- [6] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 56–61.
- [7] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

B. Software documentation

- [8] XGBoost Developers, *XGBoost Documentation*. <https://xgboost.readthedocs.io>
- [9] S. Ramírez, *FastAPI Documentation*. <https://fastapi.tiangolo.com>
- [10] Pydantic Team, *Pydantic Documentation*. <https://docs.pydantic.dev>
- [11] SQLAlchemy Project, *SQLAlchemy Documentation*. <https://docs.sqlalchemy.org>
- [12] Vercel, *Next.js Documentation*. <https://nextjs.org/docs>
- [13] Meta Open Source, *React Documentation*. <https://react.dev>
- [14] pytest Development Team, *pytest Documentation*. <https://docs.pytest.org>

C. Domain and project sources

- [15] Centre for Railway Information Systems, *National Train Enquiry System (NTES)*. <https://enquiry.indianrail.gov.in/ntes>
- [16] RailRadar, *RailRadar live train data API* (used for the optional live-data adapter, Section 13). <https://api.railradar.io>
- [17] Ministry of Education, Government of India, *Smart India Hackathon – Problem Statement 26028: Dynamic ETA Forecasting*. <https://www.sih.gov.in>
- [18] Project documents: *PRD_Dynamic_ETA_Forecasting.pdf*, *day2_dataset_report.md*, *model_training.md*, *model_evaluation.md*, *frontend_guide.md* (repository folder docs/, see Section 23).

Web resources were last accessed in September 2026.